



Universidad Nacional del Nordeste



Facultad de Ciencias Exactas y Naturales y Agrimensura

Lic. en Sistemas de Información

Proyecto de Trabajo de Campo

Ingeniería de Software 2

2026

Grupo N° 43 - Integrantes:

<u>Apellido y Nombre</u>	<u>Número de DNI</u>
Zini, Samuel Nehuen	44.196.064
Orban, Tobias Naim	46.385.637

Profesores:

Titular: Lic. María de los Ángeles Ferraro

JTP: Lic. Laura Inés Gómez Solís

Adscriptos/Auxiliares:

Doc. Andrea Airaldi Lezcano

Doc. Juan Andrés Carruthers

ÍNDICE

Capítulo 1. Introducción.....	6
1.1 Introducción.....	6
1.2 Breve estado del arte.....	6
1.3 Objetivos.....	7
1.3.1 Alcance de Funcionalidades Pretendido.....	7
Capítulo 2. Metodología.....	10
2.1. Ciclo de Vida del Proyecto.....	10
2.2. Planificación por Sprint (Scrum).....	10
2.2.1. Sprint 1: Registro y Gestión Básica de Pacientes.....	11
2.2.2. Sprint 2: Asignación y Gestión de Habitaciones.....	11
2.2.3. Sprint 3: Gestión de Egreso de Pacientes.....	12
2.2.4. Sprint 4: Implementación del Historial Clínico.....	12
2.2.5. Sprint 5: Integración y Validación del Sistema.....	12
2.2.6. Asignación de Roles.....	15
2.3 Estimación de Costos y Recursos.....	16
2.3.1 Product Backlog.....	16
2.3.2. Estimación de Esfuerzo por Actividad.....	19
2.3.3. Recursos Necesarios.....	20
2.4. Plan de Riesgos.....	21
2.4.1. Identificación de Riesgos.....	21
2.4.2. Análisis de Riesgos.....	21
2.4.3. Estrategias de Mitigación.....	22
2.4.4. Monitoreo de Riesgos.....	23
2.5 Modelado de sistemas.....	24
2.5.1 Diagrama de casos de uso.....	24
2.5.2 Conversación y diagrama de secuencia.....	25
2.5.2.1 Caso de uso: Registrar paciente.....	25
2.5.2.2 Caso de uso: Ver lista de pacientes.....	29
2.5.2.3 Caso de uso: Desactivar paciente.....	31
2.5.2.4 Caso de uso: Registrar evento clínico.....	34
2.5.2.5 Caso de uso: Visualizar línea de tiempo de historial clínico.....	37
2.5.2.6 Caso de uso: Asignar habitación.....	40
2.5.3. Contrato Operación Crítica.....	43
2.5.3.1 Contrato Operacion Registrar paciente.....	43
2.5.3.2 Contrato Operación Asignar Habitación.....	44
2.6. Base de datos.....	46
2.6.1. Modelo lógico: diagrama de entidad relación.....	46
2.6.2. Diccionario de datos.....	46
2.7. Diagrama de Clases.....	57
Capítulo 3. Herramientas y/o lenguajes de programación.....	58
3.1. Evaluación de Arquitecturas, Selección y Justificación.....	58

3.1.1	Arquitectura por Capas.....	58
3.1.2	Patrones de diseño aplicados.....	59
3.1.2.1	Patrón Fachada.....	60
3.1.2.2	Patrón Builder.....	61
3.2.	Herramientas utilizadas.....	63
3.2.1.	Gestión de Proyecto y Colaboración.....	64
3.2.2.	Diseño y Modelado (UML).....	65
3.2.3.	Stack Tecnológico de Desarrollo.....	65
3.2.4.	Infraestructura y Persistencia.....	66
3.2.5.	Convenciones y Estándares de Calidad.....	66
Capítulo 4.	Resultados.....	67
4.1.	Funcionalidad Desarrolladas.....	67
4.1.1.	Presentación de la Interfaz: Módulo de Gestión de Pacientes.....	67
4.1.1.1	Pantalla Principal - Listado de Pacientes (Dashboard).....	67
4.1.1.2	Formulario de Alta/Modificación de Paciente.....	68
4.1.1.3	Acción de Baja Lógica (Soft Delete).....	70
4.1.1.4	Listado de Pacientes Desactivados.....	70
4.1.2.	Presentación de la Interfaz: Módulo de Historial Clínico.....	71
4.1.2.1	Pantalla Principal - Listado de Pacientes (Dashboard).....	71
4.1.2.2	Pantalla de informacion del Historial del Paciente.....	72
4.1.2.3	Formulario para registrar nuevo evento clinico.....	73
4.1.2.4	Linea de tiempo de Historial Clinico.....	74
4.1.3.	Presentación de la Interfaz: Asignación y Gestión de Habitaciones.....	75
4.1.3.1	Pantalla de Habitaciones.....	75
4.1.3.2	Formulario para asignar Habitacion a un Paciente.....	76
4.1.4.	Presentación de la Interfaz: Panel de Administración de Usuarios.....	76
4.1.4.1	Pantalla Principal - Listado de Usuarios (Dashboard).....	77
4.1.4.2	Pantalla para generar invitacion de registro a un nuevo usuario.....	77
4.2.	Aspectos Técnicos del Backend y Persistencia.....	78
4.2.1.	Lógica de Negocio y Desacoplamiento (DTOs).....	78
4.2.2.	Estrategia de Persistencia y Integridad.....	78
4.2.3.	Implementación de Baja Lógica (Soft Delete).....	79
4.2.4.	Implementación del Historial Clínico.....	79
4.2.5.	Implementación de Internaciones y Habitaciones.....	79
4.2.6.	Implementación de Seguridad, Roles e Invitaciones.....	80
4.3.	Casos de prueba.....	80
4.3.1	Casos de prueba de funcionalidad “Login”.....	80
4.3.2	Casos de prueba de funcionalidad “Registrar Paciente”.....	82
4.3.3	Casos de prueba de funcionalidad “Asignar Habitaciones”.....	84
4.4.	Pruebas Unitarias.....	85
4.4.1	Prueba unitaria: Método: crearPaciente(PacienteRequestDTO dto, Integer idUsuario).....	85
4.4.2	Prueba unitaria: Método: internarPaciente(Integer idHabitacion,	

InternacionRequestDTO dto, Integer idUsuario).....	89
Capítulo 5. Conclusiones y futuros trabajos.....	91
5.1 Conclusiones.....	91
5.2 Aprendizajes obtenidos.....	92
5.3 Dificultades encontradas.....	92
5.4 Futuros trabajos.....	93
Bibliografía consultada.....	94

Tablas:

Tabla 1:	Product Backlog.....	16
Tabla 2:	Tabla Estimación de Esfuerzo por actividad.....	18
Tabla 3:	Análisis de Riesgos.....	19
Tabla 4:	Estrategias de mitigación.....	21
Tabla 5:	Contrato de operaciones críticas C.U. Registrar Paciente...42	
Tabla 6:	Contrato de operaciones críticas C.U. Asignar Habitación..	43
Tabla 7:	Diccionario de la tabla "Persona".....	45
Tabla 8:	Diccionario de la tabla "Provincia".....	45
Tabla 9:	Diccionario de la tabla "Localidad".....	45
Tabla 10:	Diccionario de la tabla "Domicilio".....	46
Tabla 11:	Diccionario de la tabla "Residencia".....	46
Tabla 12:	Diccionario de la tabla "Obra Social".....	46
Tabla 13:	Diccionario de la tabla "Afiliacion Obra Social".....	47
Tabla 14:	Diccionario de la tabla "Ficha Medica".....	47
Tabla 15:	Diccionario de la tabla "Ficha Alergia".....	48
Tabla 16:	Diccionario de la tabla "Alergia".....	48
Tabla 17:	Diccionario de la tabla "Ficha Enfermedad Cronica".....	48
Tabla 18:	Diccionario de la tabla "Enfermedad Cronica".....	49
Tabla 19:	Diccionario de la tabla "Ficha Antecedente Familiar".....	49
Tabla 20:	Diccionario de la tabla "Antecedente Familiar".....	49
Tabla 21:	Diccionario de la tabla "Paciente".....	50
Tabla 22:	Diccionario de la tabla "Contacto Emergencia".....	50
Tabla 23:	Diccionario de la tabla "Historial Medico".....	51
Tabla 24:	Diccionario de la tabla "Habitacion Internacion".....	51
Tabla 25:	Diccionario de la tabla "Internacion".....	52
Tabla 26:	Diccionario de la tabla "Tipo Procedimiento".....	52
Tabla 27:	Diccionario de la tabla "Rol".....	52
Tabla 28:	Diccionario de la tabla "Usuario".....	53
Tabla 29:	Diccionario de la tabla "Registro Clinico".....	54
Tabla 30:	Diccionario de la tabla "Telefono".....	54
Tabla 31:	Diccionario de la tabla "Invitacion Registro".....	55
Tabla 32:	Casos de prueba de funcionalidad "Login".....	68
Tabla 33:	Casos de prueba de funcionalidad "Registrar Paciente".....	70
Tabla 34:	Casos de prueba de funcionalidad "Asignar Habitaciones".	71

Figuras:

Figura 1: Diagrama de casos de uso.....	23
Figura 2: Diagrama de secuencia C.U. Registrar Paciente.....	28
Figura 3: Diagrama de secuencia C.U. Ver Lista de Pacientes.....	30

Figura 4: Diagrama de secuencia C.U. Desactivar Paciente.....	32
Figura 5: Diagrama de secuencia C.U. Registrar Evento Clínico.....	36
Figura 6: Diagrama de secuencia C.U. Visualizar Línea de Tiempo de Historial Clínico..	38
Figura 7: Diagrama de secuencia C.U. Asignar Habitación.....	41
Figura 8: Diagrama Entidad Relación.....	45
Figura 9: Diagrama de Arquitectura de Capas.....	58
Figura 10: Diagrama representativo de los patrones Fachada y Builder aplicados en Clinicks.....	62
Figura 11: Captura de pantalla de Sprints de Jira.....	63
Figura 12: Captura de pantalla de Calendario de Jira.....	63
Figura 13: Captura de pantalla del Repositorio de Github.....	64
Figura 14: Captura de pantalla del Dashboard con lista de pacientes, filtros y opción 'Ver Desactivados'.....	67
Figura 15: Captura de pantalla del formulario de registro de paciente, mostrando DNI, Nombre, Apellido, Fecha de Nacimiento.....	67
Figura 16: Captura de pantalla del formulario de registro de paciente, mostrando Alergias, Enfermedades Crónicas, Antecedentes Familiares, Observaciones, Contacto de Emergencias y Obra Social.....	68
Figura 17: Captura de pantalla de la alerta de confirmación previa a la baja lógica.....	69
Figura 18: Captura de pantalla Lista de Pacientes Desactivados.....	69
Figura 19: Captura de pantalla del listado de pacientes con historial médico.....	70
Figura 19: Captura de pantalla del módulo de Historial Clínico del paciente.....	71
Figura 20: Captura de pantalla del formulario para registrar un nuevo evento clínico..	72
Figura 21: Captura de pantalla de la línea de tiempo del Historial Clínico.....	73
Figura 22: Captura de pantalla del listado de habitaciones por piso y estado.....	74
Figura 23: Captura de pantalla de la asignación de habitación a un paciente.....	75
Figura 24: Captura de pantalla del Panel de Administración de Usuarios.....	76
Figura 25: Captura de pantalla de la creación de invitación para registro de usuario...	76

Capítulo 1. Introducción

1.1 Introducción

El presente trabajo tiene como finalidad el análisis, diseño e implementación de **Clinicks**, un sistema web orientado a la gestión hospitalaria, centrado principalmente en la administración de pacientes, habitaciones, internaciones e historial clínico. El proyecto surge a partir de la necesidad de contar con una solución informática que permita organizar de manera centralizada la información generada durante la atención y seguimiento de pacientes dentro de una institución de salud.

En muchos entornos hospitalarios, la gestión de pacientes puede depender de registros dispersos, procesos manuales o herramientas no integradas, lo que dificulta la consulta rápida de información, la trazabilidad de acciones y la coordinación entre los distintos roles involucrados. Frente a esta problemática, Clinicks propone una plataforma que permita registrar pacientes, administrar sus datos personales y médicos relevantes, asignar habitaciones, controlar internaciones y conservar un historial clínico ordenado cronológicamente.

El sistema está orientado al uso interno del personal autorizado del hospital, contemplando perfiles como administrador, médico, enfermero y personal administrativo. Cada rol accede a funcionalidades específicas según sus responsabilidades, favoreciendo una gestión más ordenada, segura y trazable de la información.

1.2 Breve estado del arte

En los últimos años, los sistemas de información aplicados al ámbito de la salud han adquirido una importancia creciente debido a la necesidad de mejorar la calidad de atención, optimizar los procesos internos y garantizar la disponibilidad de información confiable para la toma de decisiones. La Organización Mundial de la Salud sostiene que las iniciativas de salud digital deben apoyarse en estrategias sólidas que integren recursos tecnológicos, humanos, financieros y organizacionales para aportar valor real a los sistemas de salud.

Dentro de este contexto, los Sistemas de Información Hospitalaria permiten informatizar procesos vinculados con la admisión de pacientes, la gestión de camas, la administración de turnos, el registro de prestaciones y el acceso a información clínica. Su implementación favorece la reducción de errores administrativos, mejora la disponibilidad de datos y facilita el seguimiento de la atención brindada.

Uno de los componentes más relevantes de estos sistemas es la historia clínica electrónica, entendida como un registro organizado de la información sanitaria del paciente. Este tipo de registro permite conservar antecedentes, diagnósticos,

procedimientos, evolución clínica e intervenciones realizadas por profesionales de la salud. Además, aporta trazabilidad, ya que permite identificar cuándo se registró un evento y qué usuario fue responsable de dicha acción.

A nivel internacional, también se destaca la importancia de la interoperabilidad entre sistemas de salud. Estándares como HL7 FHIR proponen mecanismos para el intercambio electrónico de información sanitaria mediante recursos estructurados, facilitando que distintos sistemas puedan compartir datos clínicos de forma más consistente. Si bien Clinicks no implementa una integración externa con otros sistemas, su diseño contempla una estructura modular y organizada que podría facilitar futuras ampliaciones.

En Argentina, el desarrollo de sistemas que manipulan información de pacientes debe considerar el marco normativo vinculado con la protección de datos personales y los derechos del paciente. La Ley 25.326 regula el tratamiento de datos personales asentados en archivos, registros y bases de datos, mientras que la Ley 26.529 establece derechos relacionados con la historia clínica, el consentimiento informado y la relación entre paciente y profesionales de la salud. Por este motivo, el sistema propuesto debe priorizar la seguridad, la integridad de la información y el acceso restringido según roles.

En síntesis, el estado actual de los sistemas de gestión hospitalaria muestra una tendencia hacia soluciones web, centralizadas, seguras y con trazabilidad de operaciones. Clinicks se ubica dentro de esta línea, proponiendo una solución académica aplicada a un caso práctico, con foco en la gestión de pacientes, habitaciones e historial clínico.

1.3 Objetivos

1.3.1 Alcance de Funcionalidades Pretendido

Funcionalidad 1: Gestión de Pacientes

El sistema permitirá la gestión integral de los pacientes, abarcando desde su registro inicial hasta la actualización de su estado dentro del hospital.

Alcance:

- Registrar nuevos pacientes con sus datos personales (nombre, apellido, DNI, dirección, teléfono, fecha de nacimiento).
- Administrar información de domicilio detallada (localidad, provincia, tipo de residencia).
- Gestionar datos de cobertura de salud (obra social, número de credencial, vigencia).

- Centralizar antecedentes médicos críticos (grupo sanguíneo, alergias, enfermedades crónicas, antecedentes familiares).
- Registrar contactos de emergencia.
- Registrar el ingreso y egreso del paciente al hospital (fechas y horas).
- Asignar y reasignar habitaciones y pisos a pacientes internados.
- Realizar el seguimiento de traslados internos.
- Visualizar el estado de disponibilidad de habitaciones (disponibles, ocupadas, mantenimiento).
- Buscar y consultar pacientes mediante criterios como nombre, apellido o DNI.
- Realizar bajas administrativas de registros que preservan la integridad de la información histórica.

Límites:

- No se contemplan funcionalidades de facturación ni gestión de pagos a terceros.
- No incluye interacción directa con pacientes (uso exclusivo del personal del hospital).

Funcionalidad 2: Historial Clínico del Paciente

El sistema actuará como un repositorio centralizado y seguro de toda la actividad médica realizada sobre el paciente, organizada para facilitar la toma de decisiones profesionales.

Alcance:

- Generar automáticamente un expediente único por cada paciente registrado.
- Registrar y visualizar eventos clínicos, detallando procedimientos realizados y descripciones clínicas.
- Garantizar la trazabilidad de cada registro (identificación del profesional que lo realizó).
- Mostrar el historial clínico en orden cronológico (tipo línea de tiempo) para reconstruir la evolución del paciente.
- Registrar automáticamente los momentos de creación y última modificación del expediente.

- Permitir la consulta del historial por parte de usuarios autorizados según roles.
- Clasificar las intervenciones médicas por tipo de procedimiento.

Límites:

- No se permite la edición o eliminación de registros clínicos históricos para asegurar la validez legal y ética del historial (solo consulta y carga).
- No se incluyen integraciones o sincronización con proveedores externos de servicios de diagnóstico.
- El acceso estará restringido según roles de usuario (médicos, enfermeros, administrativos).

Capítulo 2. Metodología

2.1. Ciclo de Vida del Proyecto

El ciclo de vida adoptado para el desarrollo de Clinicks es un modelo de desarrollo ágil basado en Scrum. La elección de esta metodología se justifica por las siguientes razones de la naturaleza del entorno hospitalario:

- **Iteraciones cortas con entregas continuas:** Se planificaron *sprints* de 1 semana, lo que permite entregar funcionalidades utilizables de forma temprana (por ejemplo, tener un módulo básico de admisión de pacientes funcionando desde el primer sprint).
- **Adaptabilidad a cambios:** Al estar enfocado en distintos usuarios (médicos, enfermeros, administrativos), permite reajustar funcionalidades y prioridades según el *feedback* o nuevas normativas hospitalarias.
- **Comunicación constante:** Fomenta la revisión periódica con los usuarios del hospital al final de cada *sprint*, alineando el sistema a las necesidades reales y minimizando malentendidos.
- **Priorización de valor y calidad:** Permite priorizar funcionalidades críticas (ej. registro e internaciones primero) garantizando la trazabilidad y la calidad de los datos, que son aspectos críticos en salud.

2.2. Planificación por Sprint (Scrum)

La planificación del proyecto se estructura en torno al desarrollo de dos funcionalidades fundamentales: la **Gestión de Pacientes** y el **Historial Clínico del Paciente**, consideradas como el eje central del sistema hospitalario.

Se adoptará la metodología Scrum, operando en *sprints* de **una semana** de duración. El enfoque primordial es la construcción incremental y progresiva del sistema, iniciando con la gestión básica de la información de pacientes para luego evolucionar hacia la integración del historial clínico.

2.2.1. Sprint 1: Registro y Gestión Básica de Pacientes

Objetivo: Facilitar el proceso de registro de nuevos pacientes y la administración de sus datos personales.

Historias de Usuario incluidas:

- Registrar paciente (GP-01)
- Editar información de paciente (GP-03)
- Dar Baja lógica de paciente (GP-05)

Tareas:

- Diseño y estructuración del formulario de registro.
- Implementación de la validación de campos obligatorios.
- Desarrollo del *backend* para el procesamiento y alta del paciente.
- Establecimiento de la conectividad con el sistema de gestión de bases de datos.
- Desarrollo de la funcionalidad de búsqueda por Identificación (DNI) o nombre.
- Implementación de la lógica para la edición y actualización de datos.
- Implementación de la funcionalidad de reactivación una vez desactivado un paciente

2.2.2. Sprint 2: Asignación y Gestión de Habitaciones

Objetivo:

Administrar eficientemente la asignación de unidades de internación a los pacientes que requieran hospitalización.

Historias de Usuario incluidas:

- Asignar habitación (GP-02)

Tareas:

- Desarrollo del listado de habitaciones con indicación de su disponibilidad.
- Implementación de la validación de la disponibilidad en tiempo real.
- Asignación formal de la habitación al paciente.
- Actualización del estado de la habitación en la base de datos.
- Desarrollo del manejo de excepciones y errores (ej. falta de disponibilidad).

2.2.3. Sprint 3: Gestión de Egreso de Pacientes

Objetivo:

Documentar el proceso de alta de pacientes y actualizar su estado dentro del sistema.

Historias de Usuario incluidas:

- Registrar egreso de paciente (GP-04)

Tareas:

- Registro de la fecha y hora precisas del egreso.
- Liberación automática del estado de ocupación de la habitación.
- Actualización del estado del paciente (ej. 'Alta', 'Egresado').
- Aseguramiento de la persistencia y accesibilidad del historial clínico post-egreso.

2.2.4. Sprint 4: Implementación del Historial Clínico

Objetivo:

Desarrollar la funcionalidad integral del historial clínico, permitiendo la centralización y visualización cronológica de todos los eventos médicos relevantes del paciente.

Historias de Usuario incluidas:

- Consultar historial clínico del paciente (PD-02)
- Registrar eventos clínicos (consultas, procedimientos, medicación) (PD-01)

Tareas:

- Diseño de la arquitectura y estructura de datos del historial clínico.
- Implementación del módulo de registro de nuevos eventos clínicos.
- Asociación inequívoca de cada evento con el paciente correspondiente.
- Implementación del ordenamiento cronológico de la información.
- Desarrollo de herramientas de filtrado por criterios (fecha, tipología del evento).
- Implementación de mecanismos de control de acceso basados en roles.

2.2.5. Sprint 5: Integración y Validación del Sistema

Objetivo:

Verificar la correcta integración y operatividad de las funcionalidades de gestión de pacientes y historial clínico.

Historias de Usuario incluidas:

- Visualizar línea de tiempo del historial (PD-03)
- Realizar pruebas de rendimiento (TE-01)

Tareas:

- Ejecución de pruebas integrales a nivel de sistema.

- Validación del flujo operativo completo: (registro → asignación → atención → historial → egreso).
- Identificación y corrección de defectos detectados.
- Optimización de la interfaz de usuario y la experiencia de usabilidad (*UX*).
- Realización de sesiones de validación con usuarios clave (*feedback*).

Semana	Nombre Sprint	Tareas Sprint
Abril 15 - Abril 22	Registro y Gestión de paciente	☒ Diseño y estructuración del formulario de registro. ☒ Implementación de la validación de campos obligatorios. ☒ Desarrollo del backend para el procesamiento y alta del paciente. ☒ Establecimiento de la conectividad con el sistema de gestión de bases de datos. ☒ Desarrollo de la funcionalidad de búsqueda por Identificación (DNI) o nombre. ☒ Implementación de la lógica para la edición y actualización de datos.
Mayo 11 - Mayo 18	Asignación y Gestión habitaciones	☒ Desarrollo del listado de habitaciones con indicación de su disponibilidad. ☒ Implementación de la validación de la disponibilidad en tiempo real. ☒ Asignación formal de la habitación al paciente. ☒ Actualización del estado de la habitación en la base de datos. ☒ Desarrollo del manejo de excepciones y errores (ej. falta de disponibilidad).
Mayo 18 - Mayo 25	Gestión de Egreso de Pacientes	☒ Registro de la fecha y hora precisas del egreso.

		☒ Liberación automática del estado de ocupación de la habitación. ☒ Actualización del estado del paciente (ej. 'Alta', 'Egresado'). ☒ Aseguramiento de la persistencia y accesibilidad del historial clínico post-egreso.
Mayo 25 - Junio 1	Implementación del Historial	☒ Diseño de la arquitectura y estructura de datos del historial clínico. ☒ Implementación del módulo de registro de nuevos eventos clínicos. ☒ Asociación inequívoca de cada evento con el paciente correspondiente. ☒ Implementación del ordenamiento cronológico de la información. ☒ Desarrollo de herramientas de filtrado por criterios (fecha, tipología del evento). ☒ Implementación de mecanismos de control de acceso basados en roles.
Junio 1 - Junio 8	Integración y Validación del Sistema	☒ Ejecución de pruebas integrales a nivel de sistema. ☒ Validación del flujo operativo completo: (registro → asignación → atención → historial → egreso). ☒ Identificación y corrección de defectos detectados. ☒ Optimización de la interfaz de usuario y la experiencia de usabilidad (UX). ☒ Realización de sesiones de validación con usuarios clave (feedback).

2.2.6. Asignación de Roles

Para una ejecución eficiente del proyecto Clinicks, se han distribuido las responsabilidades técnicas y de gestión de la siguiente manera:

Orban, Tobias Naim

Como Desarrollador Frontend y Diseñador UI/UX, Tobias es el responsable principal de la experiencia del usuario y la lógica del lado del cliente. Sus responsabilidades específicas incluyen el diseño de todas las interfaces de usuario para asegurar una gestión de pacientes intuitiva, la implementación de la lógica del lado del cliente utilizando el *framework* React, el consumo eficiente de las APIs desarrolladas en el *backend* y la maquetación responsiva para garantizar la accesibilidad desde distintos dispositivos.

Zini, Samuel Nehuen

Como Administrador de Base de Datos (DBA) y Responsable de QA (Garantía de Calidad), Samuel se centra en la integridad y el rendimiento de los datos. Esto abarca el diseño y la normalización del modelo relacional de la base de datos, la optimización de consultas para un acceso rápido a la información, asegurar la integridad de los datos (especialmente el historial clínico) y el diseño de casos de prueba para verificar que los ingresos, egresos y el registro de eventos se realicen sin errores.

Ambos Integrantes (Orban y Zini)

Ambos integrantes comparten los roles de **Analistas Funcionales** y **Desarrolladores Backend**. Como Analistas, se encargan de traducir las necesidades del sistema hospitalario en requerimientos técnicos claros y la creación de diagramas UML. En el *Backend*, son responsables del desarrollo de la lógica de negocio y la creación de servicios REST utilizando Java con Spring Boot.

Descripción de funciones adicionales compartidas y específicas:

- **Analista (Ambos):** Nos encargamos de traducir las necesidades del sistema hospitalario en requerimientos técnicos y diagramas UML para asegurar una comprensión profunda de los requisitos.

- **Diseñador (Tobias):** Responsable de la estética y usabilidad del sistema, asegurando que la gestión de pacientes sea intuitiva para el personal administrativo y médico.
- **Responsable de Pruebas (Samuel):** Encargado de verificar que los ingresos, egresos y el historial clínico se registren sin errores mediante pruebas de integración exhaustivas.

2.3 Estimación de Costos y Recursos

La estimación se realiza contemplando un ciclo de desarrollo de 5 *sprints* con un enfoque en la optimización de recursos humanos.

2.3.1 Product Backlog

Épica: Gestion de Pacientes				
ID	Historia de Usuario	Estimación (Esfuerzo)	Prioridad	Criterios de Aceptación
GP-01	Registrar paciente	3	Alta	● El sistema debe permitir cargar nombre, apellido, DNI, dirección y teléfono. ● Si algún dato obligatorio falta, se muestra un mensaje de error
GP-02	Asignar habitación	3	Alta	● El sistema solo debe poder asignar pacientes en habitaciones disponibles ● El sistema debe mostrar un mensaje si no hay habitaciones libres.
GP-03	Editar información de paciente	3	Media	● El sistema debe permitir buscar pacientes por nombre, apellido o DNI.

				El sistema debe permitir modificar los datos de los pacientes.
GP-04	Registrar alta de paciente	3	Alta	- El sistema debe poder registrar la fecha y hora del alta. - El sistema debe actualizar automáticamente el estado de una habitación a libre - El sistema debe mantener disponible o almacenada la información del paciente
GP-05	Dar de baja lógica a un paciente	2	Media	- El sistema debe marcar al paciente como "eliminado" sin borrarlo físicamente de la BD (actualizando deleted_at) - El paciente ya no debe aparecer en los resultados de búsqueda generales.
Épica: Historial Clínico del Paciente				
ID	Historia de Usuario	Estimación (Esfuerzo)	Prioridad	Criterios de Aceptación
PD-01	Registrar eventos clínicos	5	Alta	- El sistema debe permitir registrar consultas, procedimientos y medicación vinculados a un paciente. - Se debe identificar automáticamente al profesional (usuario) que realiza el registro.

PD-02	Consultar historial clínico	3	Alta	• El sistema debe mostrar todos los eventos médicos del paciente de forma centralizada. • El acceso debe estar restringido según el rol del usuario (médico, enfermero, administrativo).
PD-03	Visualizar línea de tiempo	3	Media	• Los eventos deben mostrarse en orden cronológico para ver la evolución del paciente. • El sistema no debe permitir la edición o eliminación de registros históricos para asegurar validez legal.
PD-04	Filtrar intervenciones médicas	2	Baja	• El sistema debe permitir filtrar el historial por tipo de procedimiento o rango de fechas.

Product Backlog

Fundamentación del Método de Estimación:

Para la cuantificación del esfuerzo en el **Product Backlog**, el equipo ha seleccionado el método de **Puntos de Historia**. Esta decisión se fundamenta en los siguientes puntos clave:

- **Alineación con Scrum:** Al adoptar un ciclo de vida ágil basado en Scrum, los Puntos de Historia nos permiten medir el esfuerzo basándonos en la complejidad, el riesgo y la incertidumbre de cada tarea, en lugar de intentar predecir tiempos exactos.

- **Abstracción de la Presión Temporal:** Esta métrica no se ve afectada por la presión del tiempo cronológico, permitiendo que el equipo promedie su velocidad real de desarrollo sprint tras sprint.
- **Gestión de la Incertidumbre:** Facilita la absorción de dudas técnicas en tareas complejas, como la lógica de gestión de camas o el historial clínico, evitando estimaciones temporales que suelen ser inexactas en etapas tempranas del proyecto.

2.3.2. Estimación de Esfuerzo por Actividad

Para cumplir con la gestión integral del proyecto, se estiman las actividades globales necesarias para el Mínimo Producto Viable (MVP), asignando el esfuerzo según los roles definidos en la sección 2.6

Actividad	Responsable Principal (Rol)	Esfuerzo Estimado (Puntos)
Análisis y Modelado de Datos	Samuel Zini (DBA / Analista)	5
Configuración Arquitectura Backend	Ambos (Backend Dev)	3
Desarrollo Interfaz (UI) y Login	Tobias Orban (Frontend / UX)	8
Lógica de Gestión de Pacientes	Ambos (Fullstack)	13
Implementación de Historial Clínico	Ambos (Fullstack)	8

Pruebas y Documentación Técnica	Samuel Zini (QA)	5
Total Esfuerzo Estimado		42 Puntos

Tabla Estimación de Esfuerzo por actividad

Fundamentación de la métrica:

Elegimos Puntos de Historia gestionados a través de Jira. Este método nos permite desvincular el esfuerzo del tiempo cronológico, permitiendo que la "Velocidad del Equipo" (puntos completados por semana) absorba imprevistos sin afectar la planificación lógica del sistema hospitalario

2.3.3. Recursos Necesarios

- **Humanos:** El equipo consta de 2 desarrolladores con roles híbridos (Analista, Frontend, Backend, DBA y QA). Se estima una dedicación de 15 a 20 horas semanales por integrante para cubrir los puntos de historia asignados a cada Sprint.
- **Tecnológicos:**
 - **Gestión:** Jira para el seguimiento de puntos y cumplimiento de la metodología Scrum.
 - **Desarrollo:** React (Frontend), Spring Boot (Backend) y PostgreSQL en Supabase (Persistencia).
 - **Colaboración:** GitHub para versionado y Lucidchart para modelado UML.

2.4. Plan de Riesgos

El plan de riesgos tiene como objetivo identificar, analizar y definir estrategias para mitigar los posibles problemas que puedan surgir durante el desarrollo del sistema.

2.4.1. Identificación de Riesgos

Se identificaron los siguientes riesgos principales:

- Cambios en los requerimientos del sistema.
- Problemas de comunicación con los usuarios.
- Subestimación del esfuerzo en funcionalidades complejas.
- Falta de experiencia técnica del equipo.
- Fallas técnicas en el entorno de desarrollo.
- Resistencia al cambio por parte del personal del hospital.
- Problemas de conexión o pérdida de datos.

2.4.2. Análisis de Riesgos

Cada riesgo fue evaluado considerando su probabilidad e impacto:

Riesgo	Probabilidad	Impacto
Cambios en los requerimientos	Alta	Alto
Subestimación del esfuerzo	Alta	Alto
Problemas de comunicación	Media	Alto
Problemas de datos o conexión	Media	Alto
Falta de experiencia técnica	Media	Moderado
Resistencia al cambio	Media	Moderado
Fallas técnicas graves	Baja	Alto

Análisis de Riesgos

2.4.3. Estrategias de Mitigación

Para cada riesgo se definen acciones preventivas:

Riesgo	Estrategia de Mitigación
Cambios en requerimientos	Reuniones periódicas con usuarios clave (Product Owner) y validación continua de funcionalidades en cada <i>sprint review</i> . Definición de un proceso formal de gestión de cambios.
Problemas de comunicación	Uso de canales de comunicación definidos (ej. reuniones diarias, plataforma de seguimiento). Documentación clara de requerimientos (Historias de Usuario) y acuerdos.
Subestimación del esfuerzo	Planificación con márgenes de tiempo (buffers) para tareas críticas. Revisión y <i>grooming</i> del <i>Product Backlog</i> antes de cada <i>sprint</i> . Uso de técnicas de estimación por consenso (ej. Planning Poker).
Falta de experiencia técnica	Capacitación del equipo en tecnologías clave. Consulta y apoyo de recursos externos (expertos, foros). Fomento del desarrollo en pares (<i>pair programming</i>).
Fallas técnicas	Uso de entornos de desarrollo y prueba separados y estables. Implementación de control de versiones (Git) y copias de seguridad periódicas del código.
Resistencia al cambio	Capacitación anticipada y continua a usuarios finales (médicos, administrativos).

Riesgo	Estrategia de Mitigación
	Diseño de interfaces simples e intuitivas centradas en la usabilidad. Involucrar a líderes de opinión del hospital en el <i>feedback</i> .
Problemas de datos	Implementación de <i>backups</i> automáticos diarios de la base de datos. Definición de políticas de control de acceso y privilegios de usuario rigurosas. Monitoreo constante de la conexión y rendimiento del servidor.

Estrategias de mitigación

2.4.4. Monitoreo de Riesgos

Los riesgos serán revisados en cada *sprint* mediante:

- Seguimiento del avance del proyecto y métricas de desempeño.
- *Feedback* continuo del equipo de desarrollo y los usuarios clave (durante la *Daily Scrum* y *Sprint Review*).
- Identificación de nuevos riesgos o cambios en la probabilidad/impacto de los riesgos existentes.

Esto permitirá ajustar las estrategias de mitigación y reducir el impacto de posibles problemas.

2.5 Modelado de sistemas

2.5.1 Diagrama de casos de uso

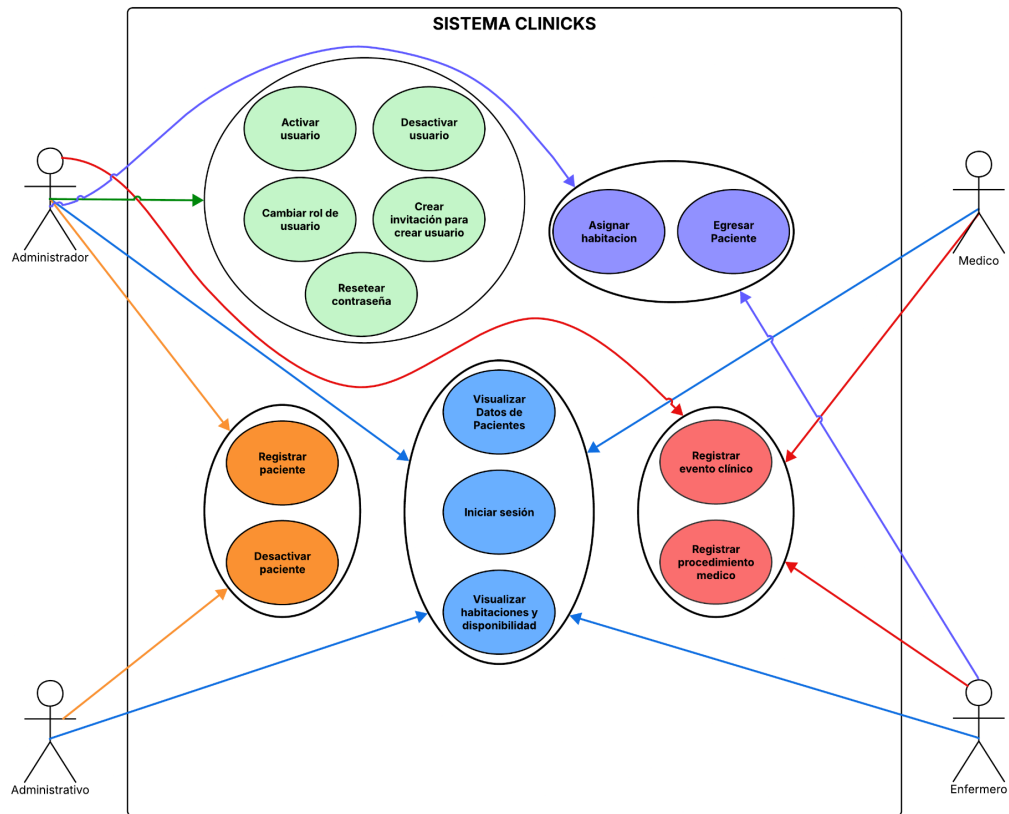


Figura 1: Diagrama de casos de uso

2.5.2 Conversación y diagrama de secuencia

2.5.2.1 Caso de uso: Registrar paciente

Actor: Administrativo

Descripción: Este caso de uso permite registrar un nuevo paciente en el sistema, cargando sus datos personales, domicilio, teléfono, obra social, ficha médica inicial y contactos de emergencia. Al registrarse correctamente, el sistema crea automáticamente el historial médico del paciente y los primeros registros clínicos administrativos.

Acción	Curso normal	Curso alternativo
1. A: Solicita al sistema comenzar con el proceso de registro de un nuevo paciente.		
2. S: Muestra el formulario de ingreso de datos del paciente.		
3. A: Ingresa nombre, apellido, DNI, fecha de nacimiento, dirección, teléfono, obra social, tipo de sangre, alergias, enfermedades crónicas, antecedentes familiares y contactos de emergencia.		
4. A: Confirma el registro del paciente.		
5. S: Envía los datos mediante <code>patientService.create(data)</code> .		
6. S: Ejecuta <code>POST /api/pacientes</code> en <code>PacienteController.crearPaciente(dto, auth)</code> .	6.1: La solicitud llega correctamente al backend.	6.2: Ocurre un error de comunicación. 6.2.1: Se muestra "Error al guardar el paciente". 6.2.2: Finaliza el caso de uso.
7. S: Identifica al usuario autenticado mediante <code>usuarioRepository.findByEmailActivo(auth.getName())</code> .	7.1: El usuario existe y está activo.	7.2: El usuario no existe. 7.2.1: Se muestra un mensaje de error. 7.2.2: Finaliza el caso de uso.

8. S: Invoca <code>pacienteService.crearPaciente(dto, idUsuario)</code> .		
9. S: Verifica que no exista un paciente con el mismo DNI mediante <code>pacienteRepository.existePorDni(dto.getDni())</code> .	9.1: El DNI no existe.	9.2: El DNI ya existe. 9.2.1: Se informa DNI duplicado. 9.2.2: Finaliza el caso de uso.
10. S: Valida afiliación y teléfonos mediante <code>validarAfiliadoUnico()</code> y <code>validarTelefonosSolicitud()</code> .	10.1: Los datos son válidos.	10.2: Existe afiliado o teléfono duplicado. 10.2.1: Se muestra el error correspondiente. 10.2.2: Finaliza el caso de uso.
11. S: Crea las entidades Persona, FichaMedica, Domicilio, Residencia y Paciente.		
12. S: Guarda el paciente mediante <code>pacienteRepository.save(paciente)</code> .	12.1: El paciente se guarda correctamente.	12.2: Ocurre un error al guardar. 12.2.1: Se muestra mensaje de error.
13. S: Carga alergias, enfermedades crónicas, antecedentes, teléfono y contactos de emergencia.		
14. S: Busca o crea el historial médico mediante <code>historialMedicoRepository.encontrarPorIdPaciente()</code> y <code>historialMedicoRepository.save()</code> .		
15. S: Registra eventos iniciales mediante <code>registrarEventoInicial()</code> : "Apertura de historial" y "Registro administrativo", si corresponde.		
16. S: Devuelve <code>PacienteResponseDTO</code> .		

17. S: Muestra “Paciente registrado correctamente” y actualiza la lista de pacientes.		
18. Fin del caso de uso.		

Diagrama de Secuencia: Registrar Paciente
Caso normal

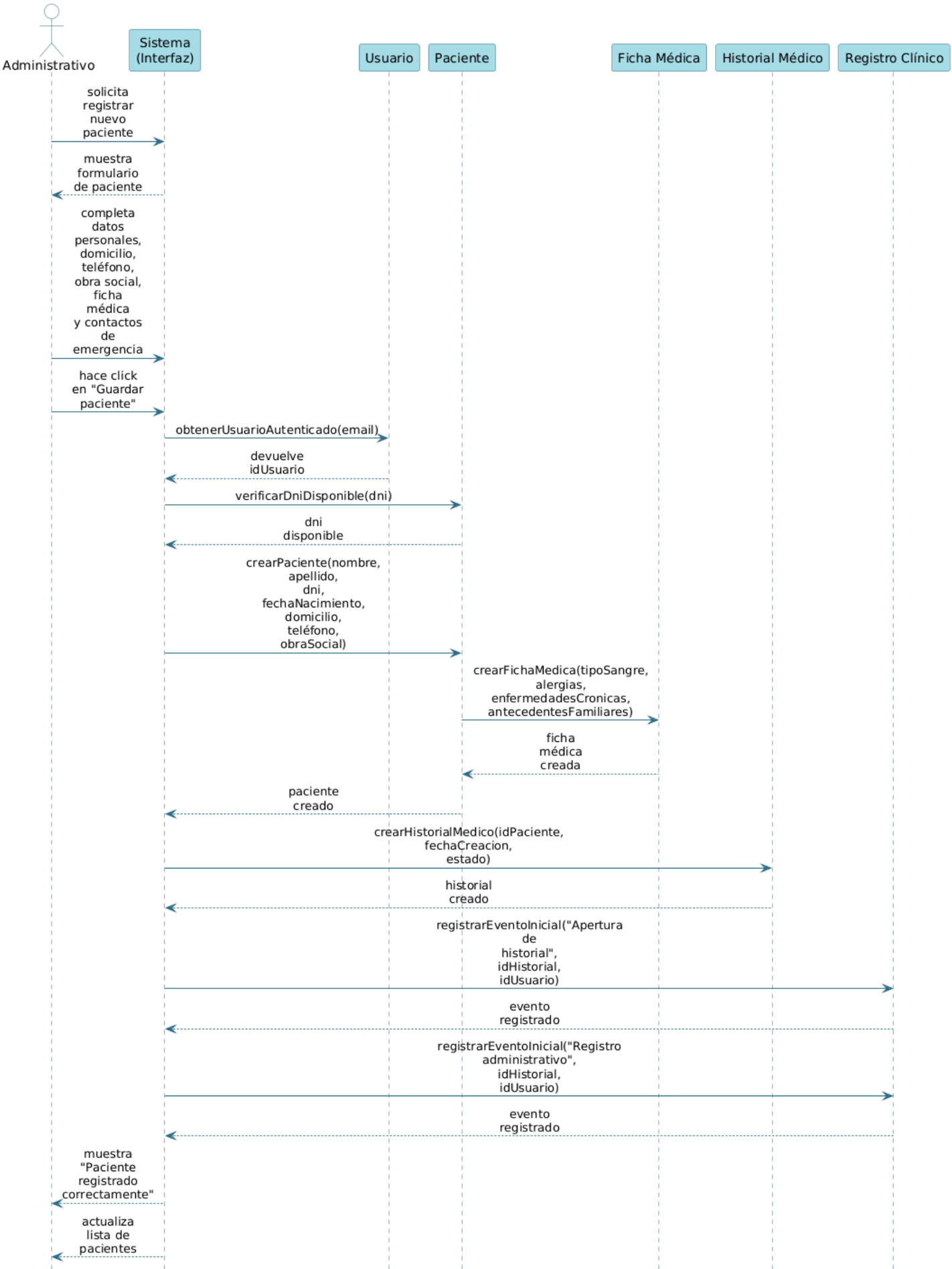


Figura 2: Diagrama de secuencia C.U. Registrar Paciente.

2.5.2.2 Caso de uso: Ver lista de pacientes

Actor: Administrativo / Médico / Enfermero

Descripción: Este caso de uso permite consultar el listado de pacientes activos registrados en el sistema, mostrando sus datos principales y permitiendo aplicar filtros por nombre, DNI, obra social o estado.

Acción	Curso normal	Curso alternativo
1. A: Accede al módulo de Pacientes.		
2. S: Solicita el listado de pacientes mediante <code>patientService.getAll()</code> .		
3. S: Ejecuta GET <code>/api/pacientes</code> en <code>PacienteController.obtenerTodosLosPacientes()</code> .		
4. S: Invoca <code>pacienteService.obtenerTodosLosPacientes()</code> .	4.1: La solicitud es procesada correctamente.	4.2: Ocurre un error de comunicación o servidor. 4.2.1: Se muestra "No se pudieron cargar los pacientes". 4.2.2: Finaliza el caso de uso.
5. S: Recupera los pacientes activos mediante <code>pacienteRepository.encontrarTodosLosPacientesActivosConDetalles()</code> .	5.1: Existen pacientes activos.	5.2: No existen pacientes activos. 5.2.1: Se muestra la tabla vacía.
6. S: Convierte los resultados a <code>PacienteResponseDTO</code> mediante <code>mapearProyeccionADTO()</code> .		
7. S: Devuelve la lista al frontend.		

8. S: Muestra la tabla de pacientes activos.		
9. A: Puede buscar por nombre o DNI, y filtrar por obra social o estado.		
10. Fin del caso de uso.		

**Diagrama de Secuencia: Ver Lista de Pacientes
Caso normal**

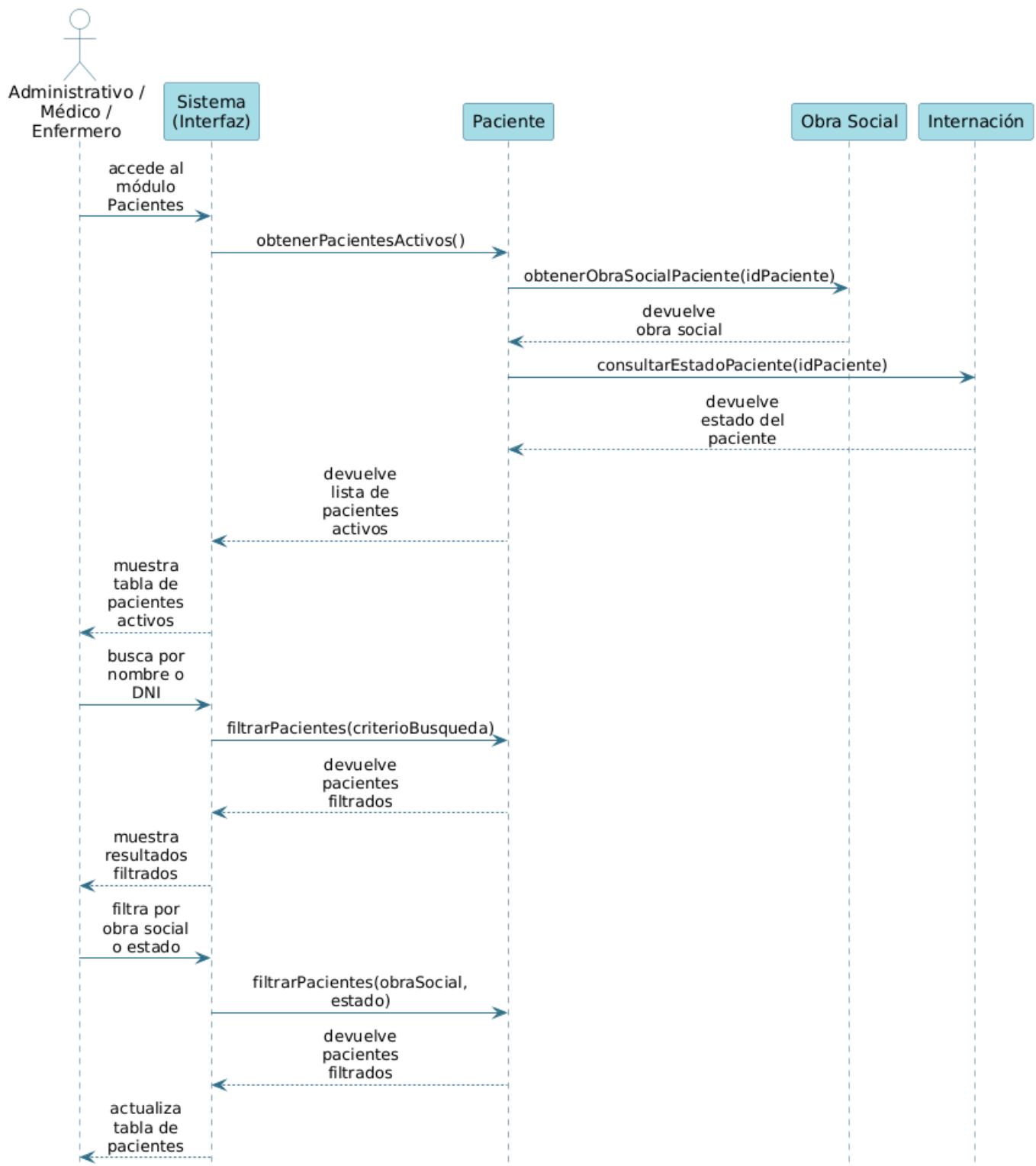


Figura 3: Diagrama de secuencia C.U. Ver Lista de Pacientes.

2.5.2.3 Caso de uso: Desactivar paciente

Actor: Administrativo

Descripción: Este caso de uso permite realizar la baja lógica de un paciente activo, conservando su información histórica en la base de datos mediante el campo deletedAt.

Acción	Curso normal	Curso alternativo
1. A: Selecciona la opción de baja sobre un paciente activo.		
2. S: Muestra un modal de confirmación indicando que el registro se mantendrá en el sistema de forma inactiva.		
3. A: Confirma la baja del paciente.		
4. S: Envía la solicitud mediante <code>patientService.delete(id)</code> .		
5. S: Ejecuta DELETE <code>/api/pacientes/{id}</code> en <code>PacienteController.eliminarPaciente(id)</code> .		
6. S: Invoca <code>pacienteService.eliminarPaciente(id)</code> .		
7. S: Busca el paciente activo mediante <code>obtenerPacienteActivo(id)</code> .	7.1: El paciente existe y se encuentra activo.	7.2: El paciente no existe o ya fue dado de baja. 7.2.1: Se muestra el error correspondiente. 7.2.2: Finaliza el caso de uso.

8. S: Verifica que el paciente no tenga una internación activa mediante <code>internacionRepository.encontrarActivaPorPaciente(id)</code> .	8.1: El paciente no se encuentra internado.	8.2: El paciente se encuentra internado. 8.2.1: Se muestra "No se puede dar de baja al paciente porque está internado/a actualmente". 8.2.2: Finaliza el caso de uso.
9. S: Asigna fecha y hora actual al campo <code>deletedAt</code> .		
10. S: Guarda el paciente mediante <code>pacienteRepository.save(paciente)</code> .	10.1: La baja lógica se registra correctamente.	10.2: Ocurre un error al guardar. 10.2.1: Se muestra mensaje de error.
11. S: Devuelve mensaje de confirmación.		
12. S: Muestra "Paciente dado de baja correctamente" y actualiza la lista.		
13. Fin del caso de uso.		

Diagrama de Secuencia: Desactivar Paciente Caso normal

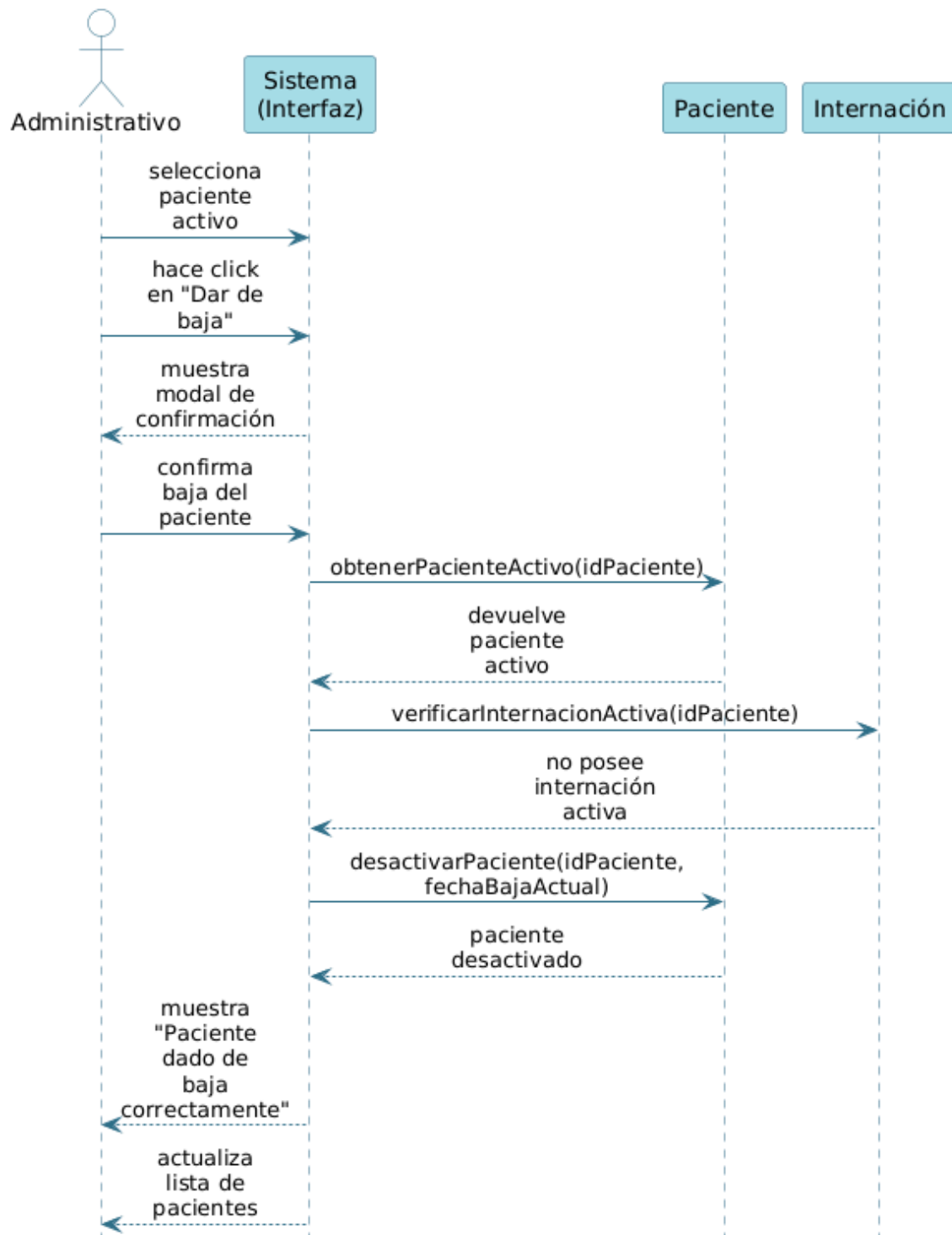


Figura 4: Diagrama de secuencia C.U. Desactivar Paciente.

2.5.2.4 Caso de uso: Registrar evento clínico

Actor: Médico / Enfermero

Descripción: Este caso de uso permite registrar un evento clínico dentro del historial médico de un paciente. El registro queda asociado al historial, al tipo de procedimiento seleccionado, al usuario profesional responsable y a la fecha/hora de carga.

Acción	Curso normal	Curso alternativo
1. A: Accede al módulo Historial.		
2. S: Muestra el listado o buscador de pacientes con historial médico.		
3. A: Selecciona un paciente.		
4. S: Obtiene el historial médico activo del paciente mediante <code>historialMedicoRepository.encontrarPorIdPaciente(idPaciente)</code> .	4.1: El historial existe y está activo.	4.2: No existe historial o no se encuentra activo. 4.2.1: Se muestra mensaje de error. 4.2.2: Finaliza el caso de uso.
5. S: Muestra la línea de tiempo y la opción de nuevo registro clínico.		
6. A: Selecciona "Nuevo registro clínico".		
7. S: Muestra el formulario con tipo de procedimiento y descripción.		
8. A: Selecciona el tipo de procedimiento e ingresa la descripción clínica.		
9. A: Confirma el registro.		

10. S: Identifica automáticamente al usuario profesional autenticado.	10.1: La sesión se encuentra activa.	10.2: La sesión expiró o el usuario no es válido. 10.2.1: Se redirige al login o se informa error. 10.2.2: Finaliza el caso de uso.
11. S: Valida que el tipo de procedimiento y la descripción estén completos.	11.1: Los datos son válidos.	11.2: Faltan datos obligatorios. 11.2.1: Se muestra alerta de validación. 11.2.2: Vuelve al paso 8.
12. S: Crea un RegistroClinico con descripción, fecha actual, historial, tipo de procedimiento y usuario responsable.		
13. S: Guarda el registro mediante registroClinicoRepository.save(registroClinico).	13.1: El registro se guarda correctamente.	13.2: Ocurre un error al guardar. 13.2.1: Se muestra mensaje de error.
14. S: Actualiza la fecha de modificación del historial médico.		
15. S: Muestra mensaje de éxito y actualiza la línea de tiempo.		
16. Fin del caso de uso.		

Diagrama de Secuencia: Registrar Evento Clínico Caso normal

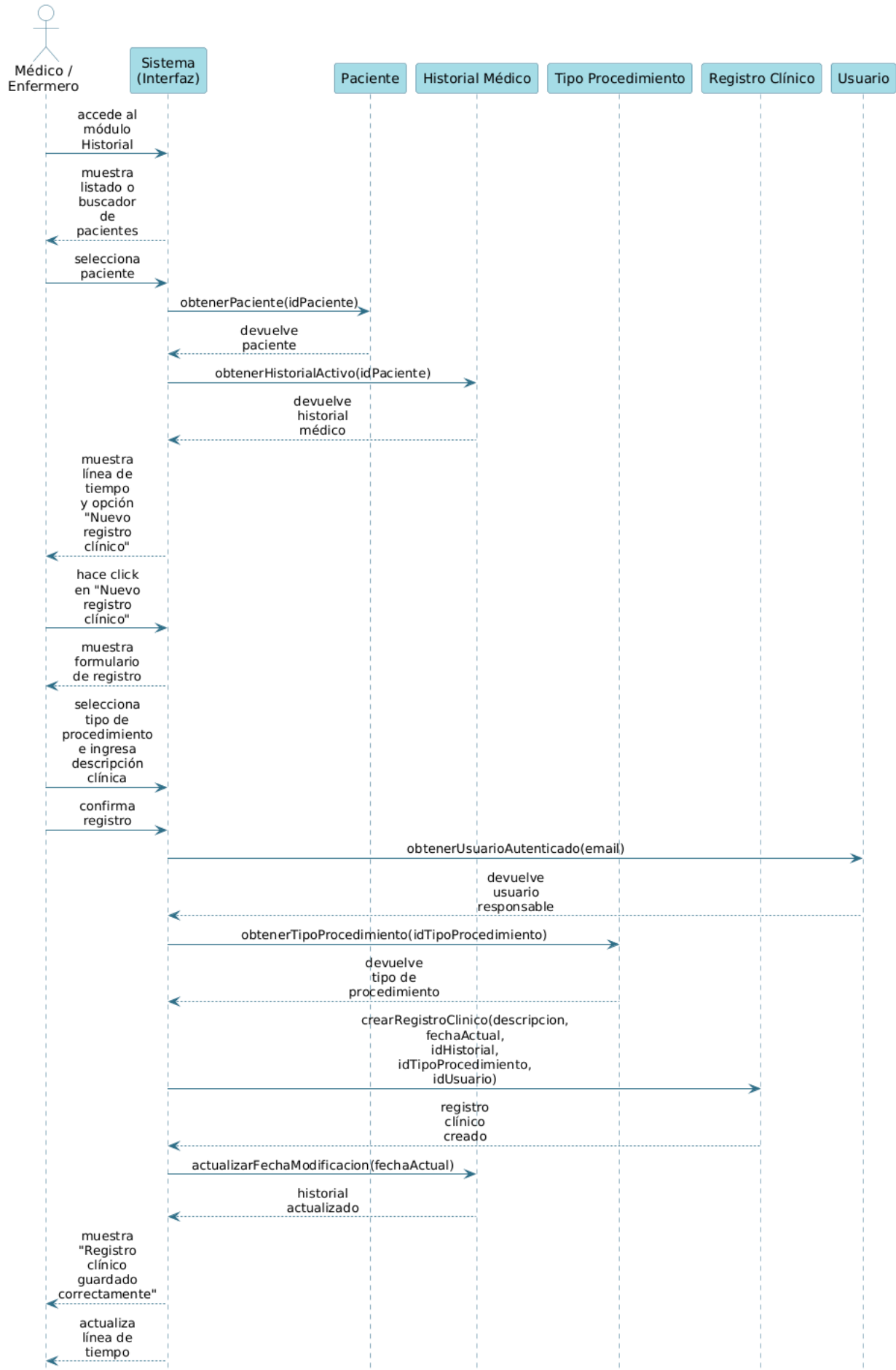


Figura 5: Diagrama de secuencia C.U. Registrar Evento Clínico.

2.5.2.5 Caso de uso: Visualizar línea de tiempo de historial clínico

Actor: Médico / Enfermero / Administrativo autorizado

Descripción: Este caso de uso permite visualizar los eventos clínicos registrados en el historial médico de un paciente, ordenados cronológicamente para reconstruir su evolución.

Acción	Curso normal	Curso alternativo
1. A: Accede al módulo Historial.		
2. S: Muestra el listado o buscador de pacientes.		
3. A: Selecciona un paciente para consultar su historial.		
4. S: Busca el historial mediante <code>historialMedicoRepository.encontrarPorIdPaciente(idPaciente)</code> .	4.1: El historial existe.	4.2: El paciente no posee historial. 4.2.1: Se muestra mensaje indicando que no hay historial disponible. 4.2.2: Finaliza el caso de uso.
5. S: Recupera los registros clínicos mediante <code>registroClinicoRepository.findAllByHistorialOrderByFechaRegistroDesc(historial)</code> .	5.1: Existen registros clínicos.	5.2: No existen registros clínicos. 5.2.1: Se muestra la línea de tiempo vacía.

6. S: Ordena y presenta los registros clínicos por fecha, tipo de procedimiento, descripción y profesional responsable.		
7. S: Muestra la línea de tiempo del historial clínico del paciente.		
8. A: Consulta la evolución clínica del paciente.		
9. Fin del caso de uso.		

**Diagrama de Secuencia: Registrar Evento Clínico
Caso normal**

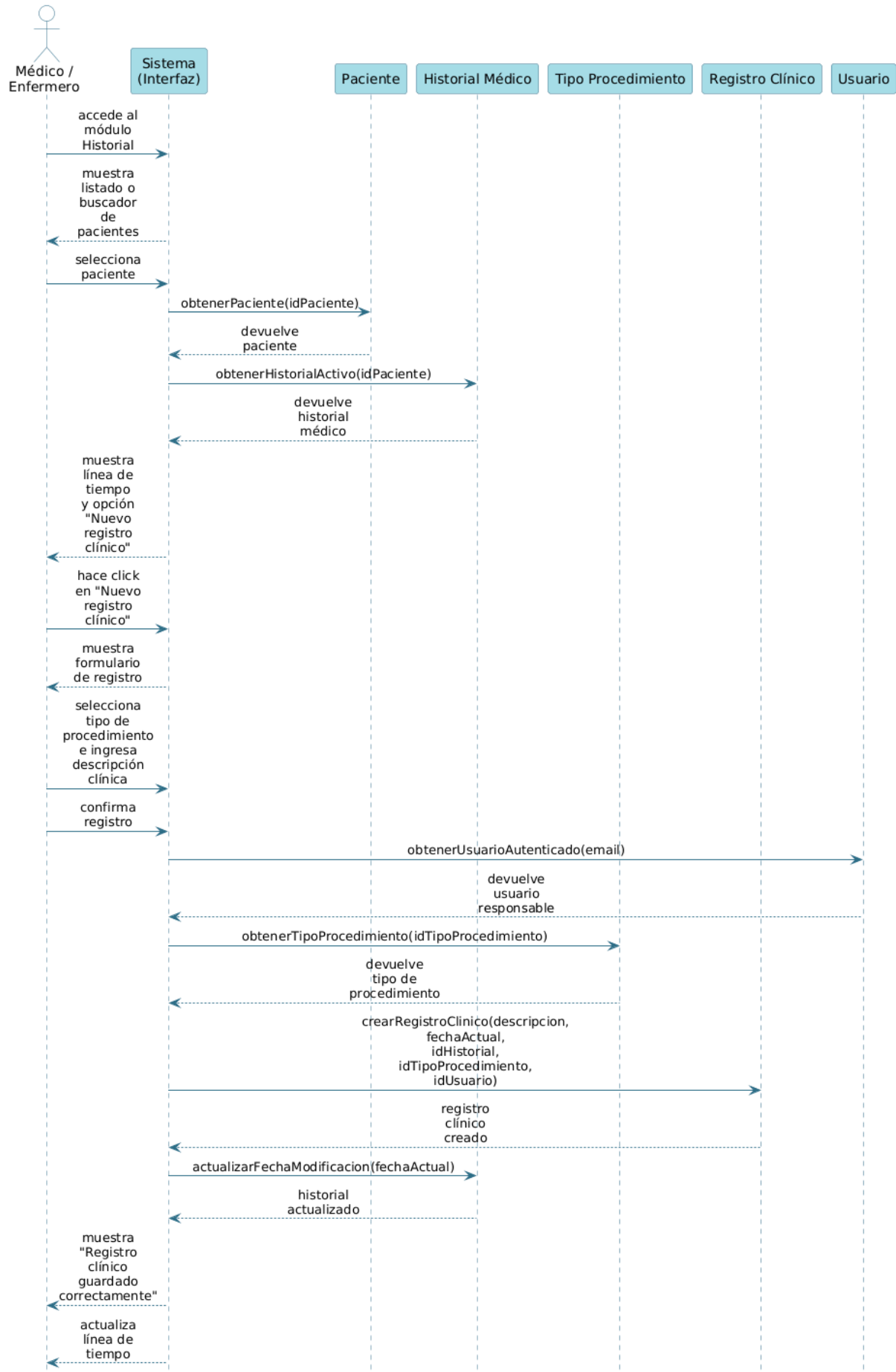


Figura 6: Diagrama de secuencia C.U. Visualizar Línea de Tiempo de Historial Clínico.

2.5.2.6 Caso de uso: Asignar habitación

Actor: Enfermero

Descripción: Este caso de uso permite asignar una habitación disponible a un paciente registrado, creando una internación activa asociada al historial médico del paciente y actualizando el estado de la habitación.

Acción	Curso normal	Curso alternativo
1. A: Accede al módulo Habitaciones.		
2. S: Solicita el listado de habitaciones mediante <code>habitacionInternacionRepository.findAll()</code> .		
3. S: Muestra las habitaciones con su estado: disponible, ocupada o mantenimiento.	3.1: Existen habitaciones disponibles.	3.2: No hay habitaciones disponibles. 3.2.1: Se muestra "No hay habitaciones disponibles". 3.2.2: Finaliza el caso de uso.
4. A: Selecciona una habitación disponible.		
5. A: Selecciona el paciente que será internado.		
6. S: Busca el paciente activo mediante <code>pacienteRepository.encontrarPacienteActivoPorId(idPaciente)</code> .	6.1: El paciente existe y está activo.	6.2: El paciente no existe o está desactivado. 6.2.1: Se muestra mensaje de error. 6.2.2: Finaliza el caso de uso.

7. S: Busca el historial médico mediante <code>historialMedicoRepository.encontrarPorIdPaciente(idPaciente)</code> .	7.1: El historial existe.	7.2: El historial no existe. 7.2.1: Se informa el error o se crea el historial si corresponde.
8. S: Verifica que la habitación seleccionada siga disponible.	8.1: La habitación está disponible.	8.2: La habitación fue ocupada o está en mantenimiento. 8.2.1: Se informa el error y vuelve al listado.
9. S: Crea una Internacion con fecha de inicio actual, historial y habitación seleccionada.		
10. S: Guarda la internación mediante <code>internacionRepository.save(internacion)</code> .	10.1: La internación se guarda correctamente.	10.2: Ocurre un error al guardar. 10.2.1: Se muestra mensaje de error.
11. S: Actualiza el estado de la habitación a ocupada.		
12. S: Guarda la habitación mediante <code>habitacionInternacionRepository.save(habitacion)</code> .		
13. S: Muestra mensaje de éxito y actualiza el mapa de habitaciones.		
14. Fin del caso de uso.		

Diagrama de Secuencia: Asignar Habitación Caso normal

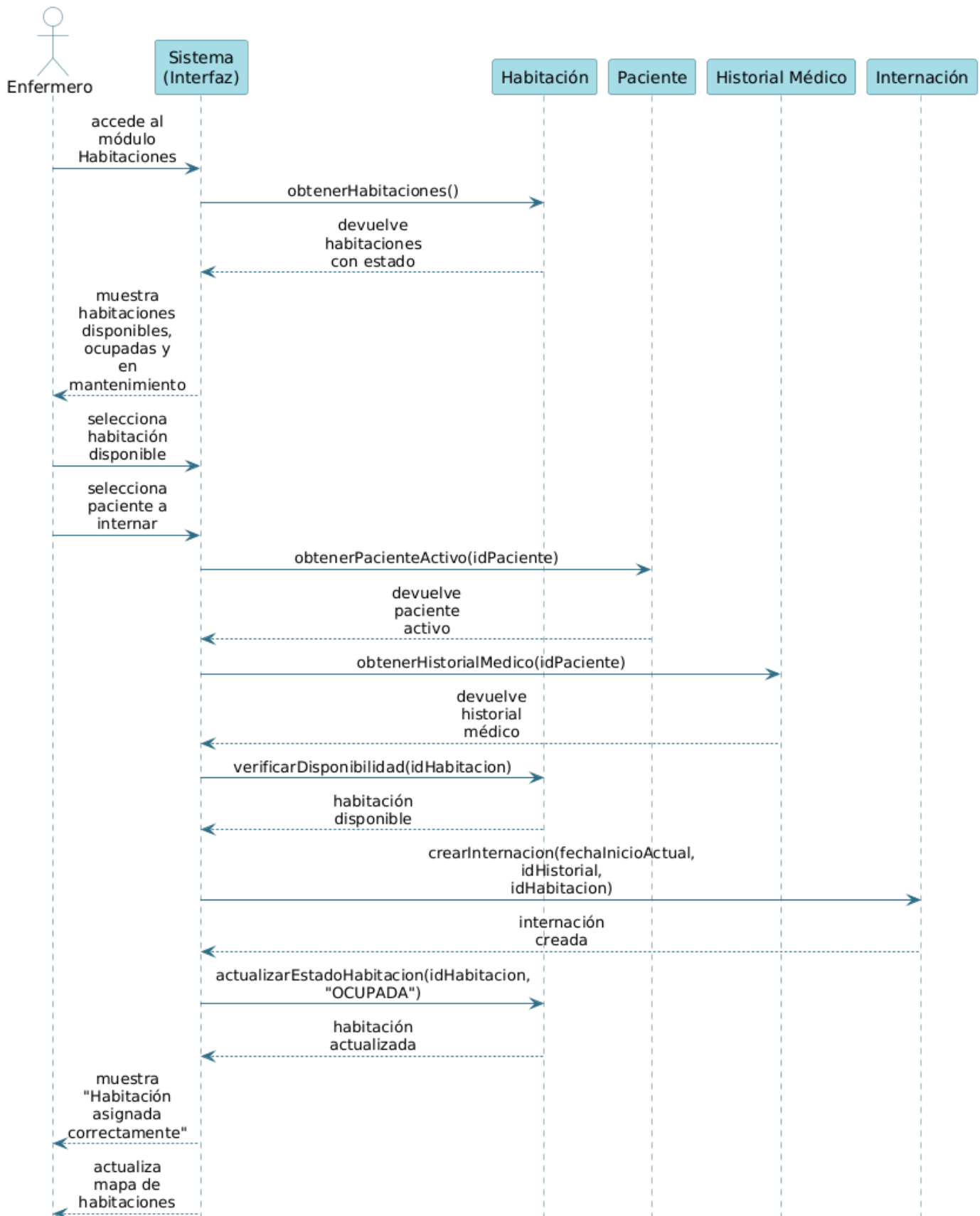


Figura 7: Diagrama de secuencia C.U. Asignar Habitación.

2.5.3. Contrato Operación Crítica

2.5.3.1 Contrato Operacion Registrar paciente

Campo	Descripción
1. Nombre	crearPaciente(dni: int, nombre: varchar, apellido: varchar, direccion: varchar, telefono: varchar, fecha_nacimiento: date)
2. Referencia Cruzada	C.U. Registrar paciente.
3. Responsabilidades	Registrar en el sistema los datos personales de un paciente que ingresa por primera vez al hospital, permitiendo que posteriormente pueda ser atendido y asociado a su historial clínico.
4. Excepciones	- Si alguno de los campos obligatorios está incompleto, el sistema muestra un mensaje indicando los campos faltantes y no permite guardar el registro. - Si el DNI ingresado ya corresponde a un paciente registrado, el sistema informa que el paciente ya existe y cancela la operación. - Si el formato de algún dato no es válido, por ejemplo teléfono, DNI o fecha de nacimiento, el sistema muestra un mensaje de error y solicita la corrección. - Si ocurre un error al guardar los datos en la base de datos, el sistema informa el error y no confirma el registro.
5. Pre-condiciones	- El administrativo se encuentra autenticado en el sistema. - El administrativo posee permisos para registrar pacientes. - El paciente no se encuentra registrado previamente en el sistema. - El sistema se encuentra disponible para acceder a la base de datos.

6. Post-condiciones	- Se creó una nueva instancia de Paciente.- Se guardaron correctamente los datos personales del paciente en la base de datos.- El paciente quedó disponible para futuras consultas, ediciones y atención médica.- Se muestra un mensaje de éxito confirmando el registro del paciente.- El sistema permite asociar posteriormente el paciente a una ficha médica o historial clínico.
----------------------------	--

Contrato de operaciones críticas C.U. Registrar Paciente

2.5.3.2 Contrato Operación Asignar Habitación

Campo	Descripción
1. Nombre	asignarHabitacion(id_paciente: int, id_habitacion: int, fecha_inicio: datetime)
2. Referencia Cruzada	C.U. Asignar Habitación.
3. Responsabilidades	Gestionar la asignación de una habitación disponible a un paciente que requiere internación, registrando el inicio de la internación y actualizando el estado de la habitación dentro del sistema.
4. Excepciones	- Si no existen habitaciones disponibles, el sistema muestra un mensaje indicando que no hay unidades disponibles y finaliza el caso de uso.- Si la habitación seleccionada fue ocupada recientemente por otro paciente, el sistema informa el error y vuelve a mostrar el listado actualizado de habitaciones.- Si el paciente no existe o no se encuentra registrado en el sistema, se informa el error y se cancela la operación.- Si ocurre un error al registrar la internación o actualizar el estado de la habitación, el sistema muestra un mensaje de error y no confirma la asignación.

5. Pre-condiciones	- El administrativo se encuentra autenticado en el sistema.- El administrativo posee permisos para asignar habitaciones.- El paciente se encuentra registrado en el sistema.- Existen habitaciones cargadas en el sistema.- La habitación seleccionada debe encontrarse en estado disponible al momento de la asignación.
6. Post-condiciones	- Se creó una nueva instancia de Internación asociada al paciente.- Se registró la fecha de inicio de la internación.- Se asoció la habitación seleccionada al paciente internado.- Se actualizó el estado de la habitación a ocupada.- Se muestra un mensaje de éxito confirmando la asignación de la habitación.

Contrato de operaciones críticas C.U. Asignar Habitación.

2.6. Base de datos

2.6.1. Modelo lógico: diagrama de entidad relación.

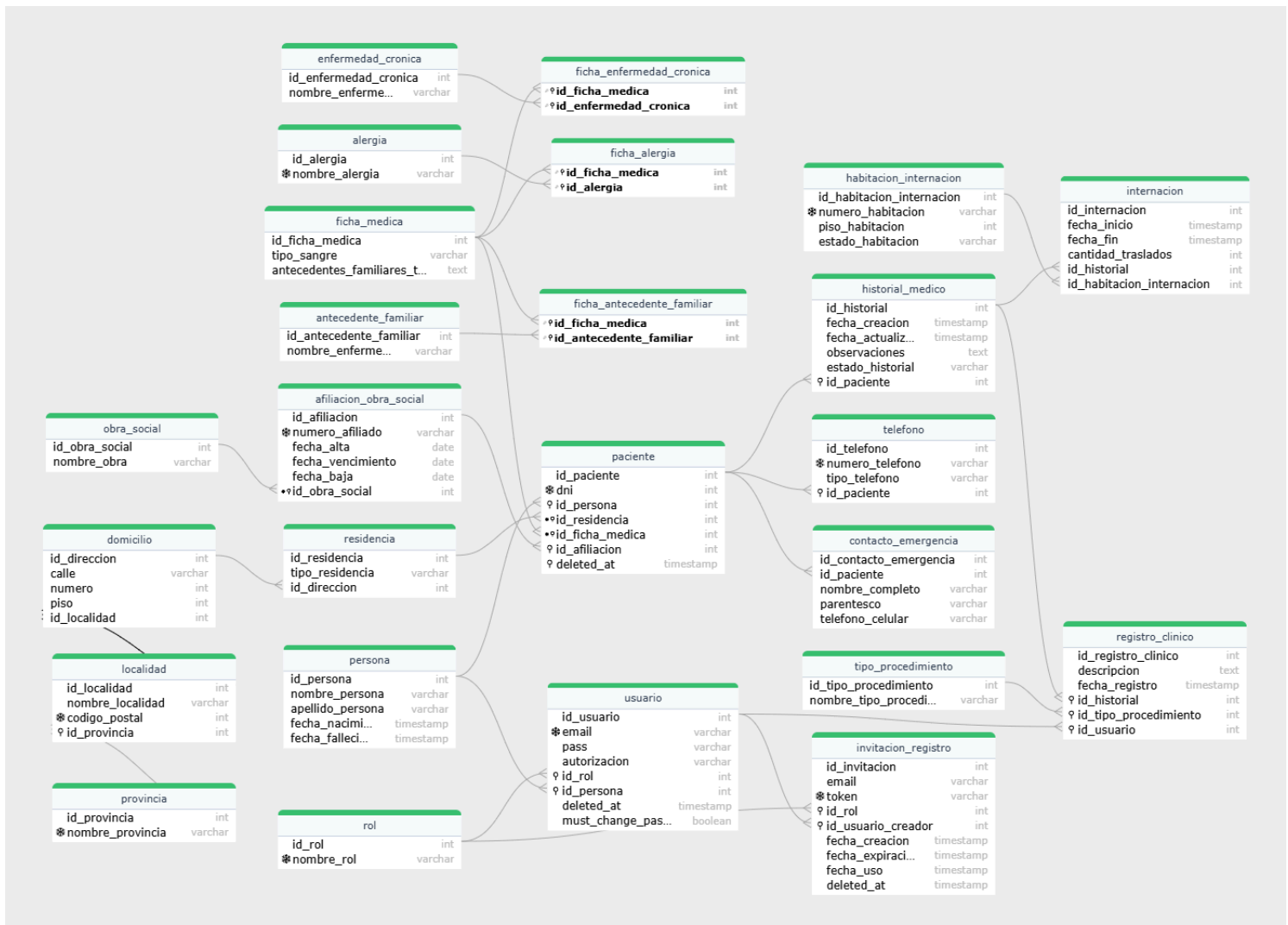


Figura 8: Diagrama Entidad Relación

2.6.2. Diccionario de datos

Convenciones de restricciones:

- PK = Clave primaria
- FK = Clave foránea (se indica la tabla referenciada)
- NN = Not Null (valor obligatorio)
- UNIQUE = Valor único en la tabla
- — = Sin restricción adicional (campo opcional)
- Sí / No en columna 'Nulo' indica si el campo acepta valores nulos

Tabla: persona

Almacena los datos personales básicos compartidos entre pacientes y usuarios del sistema.

Campo	Tipo de dato	Nul o	Restriccione s	Descripción
id_persona	INT	No	PK	Identificador único de la persona
nombre_persona	VARCHAR(200)	No	NN	Nombre/s de la persona
apellido_persona	VARCHAR(200)	No	NN	Apellido/s de la persona
fecha_nacimiento	TIMESTAMP	No	NN	Fecha de nacimiento de la persona
fecha_fallecimiento	TIMESTAMP	Si	—	Fecha de fallecimiento de la persona si fuera así

Diccionario de la tabla “Persona”

Tabla: provincia

Catálogo de provincias del país.

Campo	Tipo de dato	Nul o	Restriccione s	Descripción
id_provincia	INT	No	PK	Identificador único de la provincia
nombre_provincia	VARCHAR(200)	No	NN, UNIQUE	Nombre de la provincia (único)

Diccionario de la tabla “Provincia”

Tabla: localidad

Localidades asociadas a una provincia.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_localidad	INT	No	PK	Identificador único de la localidad
nombre_localidad	VARCHAR(200)	No	NN	Nombre de la localidad
codigo_postal	INT	No	NN, UNIQUE	Código postal (único en el país)
id_provincia	INT	No	FK → provincia	Provincia a la que pertenece la localidad

Diccionario de la tabla “Localidad”

Tabla: domicilio

Dirección física asociada a un paciente.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_direccion	INT	No	PK	Identificador único del domicilio
calle	VARCHAR(200)	No	NN	Nombre de la calle
numero	INT	No	NN	Número de la dirección
piso	INT	Sí	—	Piso del domicilio (opcional)
id_localidad	INT	No	FK → localidad	Localidad del domicilio

Diccionario de la tabla “Domicilio”

Tabla: residencia

Dirección física asociada a un paciente.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_residencia	INT	No	PK	Identificador único de la residencia
tipo_residencia	VARCHAR(15)	No	NN, CHECK	Tipo de residencia (transitorio o permanente)
id_direccion	INT	No	FK → domicilio	Domicilio donde reside el paciente

Diccionario de la tabla “Residencia”

Tabla: obra_social

Catálogo de obras sociales o coberturas médicas.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_obra_social	INT	No	PK	Identificador único de la obra social
nombre_obra	VARCHAR(100)	No	NN	Nombre de la obra social o cobertura

Diccionario de la tabla “Obra Social”

Tabla: afiliacion_obra_social

Datos de afiliación de un paciente a una obra social.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_afiliacion	INT	No	PK	Identificador único de la afiliación .
numero_afiliado	VARCHAR(50)	No	NN, UNIQUE	Numero de afiliado de la obra social del paciente
fecha_alta	DATE	No	NN	Fecha en que el paciente obtuvo la obra social
fecha_vencimiento	DATE	Si	—	Fecha de vencimiento de la obra social
fecha_baja	DATE	Si	—	Fecha en que el paciente se da de baja en la obra social
id_obra_social	INT	No	NN, FK → obra_social	Obra social al cual le pertenece la afiliación

Diccionario de la tabla “Afiliacion Obra Social”

Tabla: ficha_medica

Antecedentes médicos relevantes del paciente, incluyendo alergias, enfermedades crónicas y tipo de sangre.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_ficha_medica	INT	No	PK	Identificador único del registro de antecedentes médicos
tipo_sangre	VARCHAR(3)	No	NN, CHECK	Tipo de sangre del paciente ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-')
antecedentes_familiares_text	TEXT	Si	—	Observaciones de antecedentes familiares

Diccionario de la tabla “Ficha Medica”

Tabla: ficha_alergia

Relación N:M entre ficha médica y alergias.

Campo	Tipo de dato	Nul o	Restriccione s	Descripción
id_ficha_medica	INT	No	PK, FK	Identificador único del registro de antecedentes médicos
id_alergia	INT	No	PK, FK	Identificador único de alergia

Diccionario de la tabla "Ficha Alergia"

Tabla: alergia

Base de datos de alergias registradas hasta el momento

Campo	Tipo de dato	Nul o	Restriccione s	Descripción
id_alergia	INT	No	PK	Identificador único de alergia
nombre_alergia	VARCHAR(100)	No	NN, UNIQUE	Nombre de la alergia que padece

Diccionario de la tabla "Alergia"

Tabla: ficha_enfermedad_cronica

Relación N:M entre ficha médica y enfermedades crónicas.

Campo	Tipo de dato	Nul o	Restriccione s	Descripción
id_ficha_medica	INT	No	PK, FK	Identificador único del registro de antecedentes médicos
id_enfermedad_cronica	INT	No	PK, FK	Identificador único de enfermedad cronica

Diccionario de la tabla "Ficha Enfermedad Cronica"

Tabla: enfermedad_cronica

Base de datos de enfermedades crónicas registradas hasta el momento

Campo	Tipo de dato	Nul o	Restriccione s	Descripción
id_enfermedad_cronica	INT	No	PK	Identificador único del registro de antecedentes médicos

nombre_enfermedad	VARCHAR(200)	No	NN	Nombre de la enfermedad crónica que padece
--------------------------	--------------	----	----	--

Diccionario de la tabla "Enfermedad Cronica"

Tabla: ficha_antecedente_familiar

Relación N:M entre ficha médica y antecedentes de parientes

Campo	Tipo de dato	Nul o	Restriccione s	Descripción
id_ficha_medica	INT	No	PK, FK	Identificador único del registro de antecedentes médicos
id_antecedente_familiar	INT	No	PK, FK	Identificador único de antecedente familiar de enfermedades

Diccionario de la tabla "Ficha Antecedente Familiar"

Tabla: antecedente_familiar

Base de datos de enfermedades graves registradas por parientes de pacientes hasta el momento

Campo	Tipo de dato	Nul o	Restriccione s	Descripción
id_antecedente_familiar	INT	No	PK	Identificador único del registro de antecedentes familiares de enfermedades
nombre_enfermedad	VARCHAR(200)	No	NN	Nombre de la enfermedad grave que tuvo un pariente

Diccionario de la tabla "Antecedente Familiar"

Tabla: paciente

Datos administrativos y médicos del paciente registrado en el sistema.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_paciente	INT	No	PK	Identificador único del paciente

dni	INT	No	NN, UNIQUE	Documento Nacional de Identidad (único)
id_residencia	INT	No	FK → residencia, UNIQUE	Residencia ACTUAL del paciente.
id_persona	INT	No	FK → persona	Datos personales básicos del paciente
id_ficha_medica	INT	No	FK → ficha_medica, UNIQUE	Antecedentes médicos del paciente (1:1)
id_afiliacion	INT	Si	FK → afiliacionobra_social	Cobertura médica opcional
deleted_at	TIMESTAMP	Si	—	Fecha de borrado lógico (soft delete)

Diccionario de la tabla “Paciente”

Tabla: contacto_emergencia

Contactos de emergencia asociados a un paciente.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_contacto_emergencia	INT	No	PK	Identificador único
id_paciente	INT	No	FK → paciente , UNIQUE	Paciente al que asiste
nombre_completo	VARCHAR(200)	No	NN	Nombre del contacto
parentesco	VARCHAR(50)	No	NN	Relación con el paciente
telefono_celular	VARCHAR(25)	No	NN	Teléfono de contacto rápido

Diccionario de la tabla “Contacto Emergencia”

Tabla: historial_medico

Historial clínico del paciente. Se crea al registrar al paciente y se actualiza con cada evento médico.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
-------	--------------	------	---------------	-------------

id_historial	INT	No	PK	Identificador único del historial médico
fecha_creacion	TIMESTAMP	No	NN	Fecha en que se creó el historial
fecha_actualizacion	TIMESTAMP	No	NN	Fecha de la última actualización del historial
observaciones	TEXT	Sí	—	Observaciones generales sobre el paciente
estado_historial	VARCHAR(15)	No	NN, CHECK	Estado del historial (activo / cerrado / archivado)
id_paciente	INT	No	FK → paciente	Paciente al que pertenece el historial

Diccionario de la tabla “Historial Medico”

Tabla: habitacion_internacion

Habitaciones disponibles en el hospital para la internación de pacientes.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_habitacion_internacion	INT	No	PK	Identificador único de la habitación
numero_habitacion	VARCHAR(4)	No	NN, UNIQUE	Número o código de la habitación (único)
piso_habitacion	INT	No	NN	Piso en el que se encuentra la habitación
estado_habitacion	VARCHAR(15)	No	NN, CHECK	Estado de la habitación: 'disponible', 'ocupada', 'mantenimiento'

Diccionario de la tabla “Habitacion Internacion”

Tabla: internacion

Registro de los períodos de internación de un paciente, incluyendo habitación asignada y traslados.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_internacion	INT	No	PK	Identificador único de la internación

fecha_inicio	TIMESTAMP	No	NN	Fecha de inicio de la internación
fecha_fin	TIMESTAMP	Sí	—	Fecha de alta del paciente (nulo si sigue internado)
cantidad_traslados	INT	No	NN, DEFAULT	Cantidad de traslados de habitación durante la internación
id_historial	INT	No	FK → historial_medico	Historial clínico al que pertenece la internación
id_habitacion_internacion	INT	No	FK → habitacion	Habitación asignada al paciente

Diccionario de la tabla “Internacion”

Tabla: tipo_procedimiento

Catálogo de tipos de procedimientos médicos que pueden realizarse sobre un paciente.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_tipo_procedimiento	INT	No	PK	Identificador único del tipo de procedimiento
nombre_tipo_procedimiento	VARCHAR(200)	No	NN	Nombre descriptivo del tipo de procedimiento

Diccionario de la tabla “Tipo Procedimiento”

Tabla: rol

Roles disponibles en el sistema que determinan los permisos de acceso de cada usuario.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_rol	INT	No	PK	Identificador único del rol
nombre_rol	VARCHAR(200)	No	NN, UNIQUE	Nombre del rol (único)

Diccionario de la tabla “Rol”

Tabla: usuario

Usuarios del sistema (personal hospitalario). Cada usuario tiene un rol que define sus permisos.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_usuario	INT	No	PK	Identificador único del usuario
email	VARCHAR(200)	No	NN, UNIQUE, CHECK	Correo electrónico del usuario (credencial de acceso)
pass	VARCHAR(200)	No	NN	Contraseña del usuario almacenada
autorizacion	VARCHAR(50)	Si	—	Nivel de autorización o estado
id_rol	INT	No	FK → rol	Rol asignado al usuario
id_persona	INT	No	FK → persona	Datos personales del usuario
must_change_password	BOOLEAN	No	NN, DEFAULT FALSE	Indica si el usuario debe cambiar su contraseña al iniciar sesión.
deleted_at	TIMESTAMP	Si	—	Borrado lógico del usuario

Diccionario de la tabla “Usuario”

Tabla: registro_clinico

Registros de los procedimientos y diagnósticos realizados sobre un paciente, vinculados al historial clínico.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_registro_clinico	INT	No	PK	Identificador único del registro clínico
descripcion	TEXT	Sí	—	Descripción detallada del procedimiento realizado
fecha_registro	TIMESTAMP	No	NN	Fecha en que se realizó el registro
id_historial	INT	No	FK → historial_medico	Historial clínico al que pertenece el registro

id_tipo_procedimiento	INT	No	FK → tipo_procedimiento	Tipo de procedimiento realizado
id_usuario	INT	No	FK → usuario, UNIQUE	Usuario que realizó o registró el procedimiento

Diccionario de la tabla “Registro Clínico”

Tabla: telefono

Números de teléfono de contacto personales o de emergencia asociados al paciente.

Campo	Tipo de dato	Nul o	Restricciones	Descripción
id_telefono	INT	No	PK	Identificador único del teléfono
numero_telefono	VARCHAR(20)	No	NN, UNIQUE	Número de teléfono (único)
tipo_telefono	VARCHAR(15)	No	NN, CHECK	Define si el teléfono es ' personal ' o de ' emergencia '.
id_paciente	INT	No	NN, FK → paciente	Persona dueña del teléfono

Diccionario de la tabla “Telefono”

Tabla: invitacion_registro

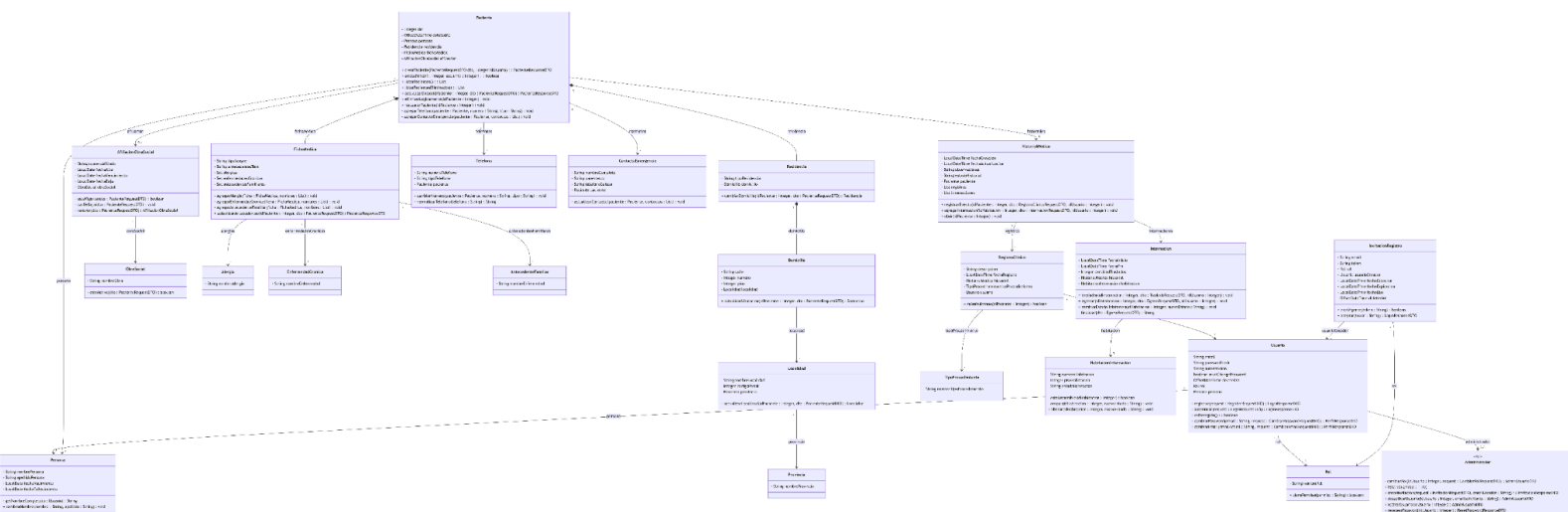
Almacena las invitaciones generadas para registrar nuevos usuarios en el sistema.

Campo	Tipo de dato	Nulo	Restricciones	Descripción
id_invitacion	INT	No	PK	Identificador único de invitacion
email	VARCHAR(200)	No	NN, CHECK	Correo electrónico del usuario invitado.
token	VARCHAR(255)	No	NN, UNIQUE	Token único utilizado para validar la invitación.
id_rol	INT	No	FK → rol	Rol asignado al usuario que se registrará con la invitación.

id_usuario_creador	INT	No	FK → usuario	Usuario que generó la invitación.
fecha_creacion	TIMESTAMP	No	NN, DEFAULT NOW()	Fecha y hora en que se creó la invitación
fecha_expiracion	TIMESTAMP	No	NN	Fecha y hora límite para usar la invitación.
fecha_uso	TIMESTAMP	No	—	Fecha y hora en que la invitación fue utilizada.
deleted_at	TIMESTAMP	Si	—	Fecha de borrado lógico de la invitación.

Diccionario de la tabla “Invitacion Registro”

2.7. Diagrama de Clases



2.8 Patrones de diseño aplicados

En el desarrollo del sistema Clinicks se aplicaron patrones de diseño con el objetivo de mejorar la organización interna del código, reducir el acoplamiento entre componentes y favorecer la mantenibilidad del sistema. En particular, se utilizaron los patrones Fachada y Builder dentro de la funcionalidad de Historial Clínico, debido a que esta funcionalidad requiere coordinar varias entidades, servicios, repositorios y reglas de negocio.

La aplicación de estos patrones complementa la arquitectura por capas adoptada para el sistema, ya que permite mantener separadas las responsabilidades entre la capa de presentación, la capa de lógica de negocio y la capa de persistencia. De esta manera, se evita que los controladores conozcan detalles internos del proceso y se mejora la claridad de la implementación.

2.8.1 Patrón Fachada

Para el desarrollo del sistema se aplicó el patrón de diseño Fachada en aquellas funcionalidades que requieren coordinar varias operaciones internas del sistema. Este patrón permite ofrecer una interfaz simple hacia procesos complejos, evitando que la capa de presentación conozca directamente todos los servicios, repositorios y reglas de negocio involucradas.

La elección de este patrón se fundamenta en la necesidad de mantener una separación clara entre la capa de presentación y la capa de lógica de negocio. En la funcionalidad de Historial Clínico, operaciones como registrar un paciente, generar un historial médico, registrar eventos clínicos iniciales o asociar tipos de procedimiento no representan acciones aisladas, sino procesos compuestos por varias tareas relacionadas.

Por este motivo, centralizar la coordinación de dichas tareas en una clase o servicio principal permite mejorar la organización del código, reducir el acoplamiento entre componentes y facilitar el mantenimiento del sistema.

Aplicación en el sistema:

En Clinicks, el patrón Fachada se aplica en la capa de servicios, principalmente en la clase encargada de coordinar la lógica de pacientes e historial clínico. Por ejemplo, desde el controlador PacienteController, al registrar un paciente se delega la operación en el servicio correspondiente, sin que el controlador tenga que conocer todos los pasos internos necesarios para completar el proceso.

Internamente, el servicio coordina distintos repositorios y entidades, como paciente, obra social, teléfono, alergia, historial médico, registro clínico, tipo de procedimiento, usuario e internación. De esta forma, el controlador no interactúa directamente con múltiples repositorios ni conoce los detalles de construcción del historial clínico.

Ejemplo aplicado:

Registrar paciente implica:

- Validar los datos ingresados.
- Crear o asociar los datos personales del paciente.
- Registrar domicilio y contacto de emergencia.
- Asociar obra social, si corresponde.
- Crear la ficha médica inicial.
- Generar el historial clínico del paciente.
- Guardar la información mediante los repositorios correspondientes.

Beneficios:

- Reduce el acoplamiento entre la capa de presentación y la lógica interna del sistema.

- Evita duplicación de código en operaciones complejas.
- Facilita el mantenimiento, ya que los cambios internos no afectan directamente al controlador.
- Mejora la trazabilidad de las operaciones críticas.
- Permite que el código sea más claro, ordenado y fácil de probar.

2.8.2 Patrón Builder

Se aplicó el patrón de diseño Builder para la construcción de objetos complejos dentro del sistema. Este patrón permite crear objetos paso a paso, de manera más ordenada y legible, evitando el uso de constructores extensos con muchos parámetros.

La elección de este patrón se fundamenta en que varias entidades del sistema Clinicks contienen múltiples atributos y relaciones. En particular, dentro de la funcionalidad de Historial Clínico, entidades como HistorialMedico y RegistroClinico poseen relaciones obligatorias con paciente, usuario, tipo de procedimiento, fecha y descripción.

Construir este tipo de objetos mediante constructores tradicionales puede generar código difícil de leer, mantener y modificar.

Aplicación en el sistema:

En Clinicks, el patrón Builder se utiliza para ordenar la creación de objetos relacionados con el historial clínico y los registros clínicos. Esto permite construir entidades del dominio de forma progresiva, indicando claramente qué datos se están incorporando en cada paso.

Por ejemplo, al generar un registro clínico inicial, el sistema debe definir una descripción, una fecha de registro, el historial médico asociado, el tipo de procedimiento correspondiente y el usuario responsable. El uso de Builder permite crear este objeto de forma clara y controlada, reduciendo errores en la asignación de atributos.

También se aplica al crear el historial médico del paciente, ya que dicho objeto debe vincularse con el paciente correspondiente, definir su estado y registrar fechas de creación o actualización.

Ejemplo aplicado:**Construcción de un paciente:**

- Cargar datos personales.
- Asociar domicilio.
- Agregar teléfono o contacto de emergencia.
- Asociar obra social.
- Cargar ficha médica.
- Incorporar alergias, enfermedades crónicas o antecedentes.
- Generar el objeto final para ser guardado.

Beneficios:

- Evita constructores largos y difíciles de interpretar.
- Mejora la legibilidad del código.
- Reduce errores en el orden o asignación de parámetros.
- Facilita la incorporación de nuevos atributos en futuras versiones.
- Permite construir objetos complejos de manera controlada y mantenible.

Funcionalidad Historial Clínico - Patrones Fachada y Builder

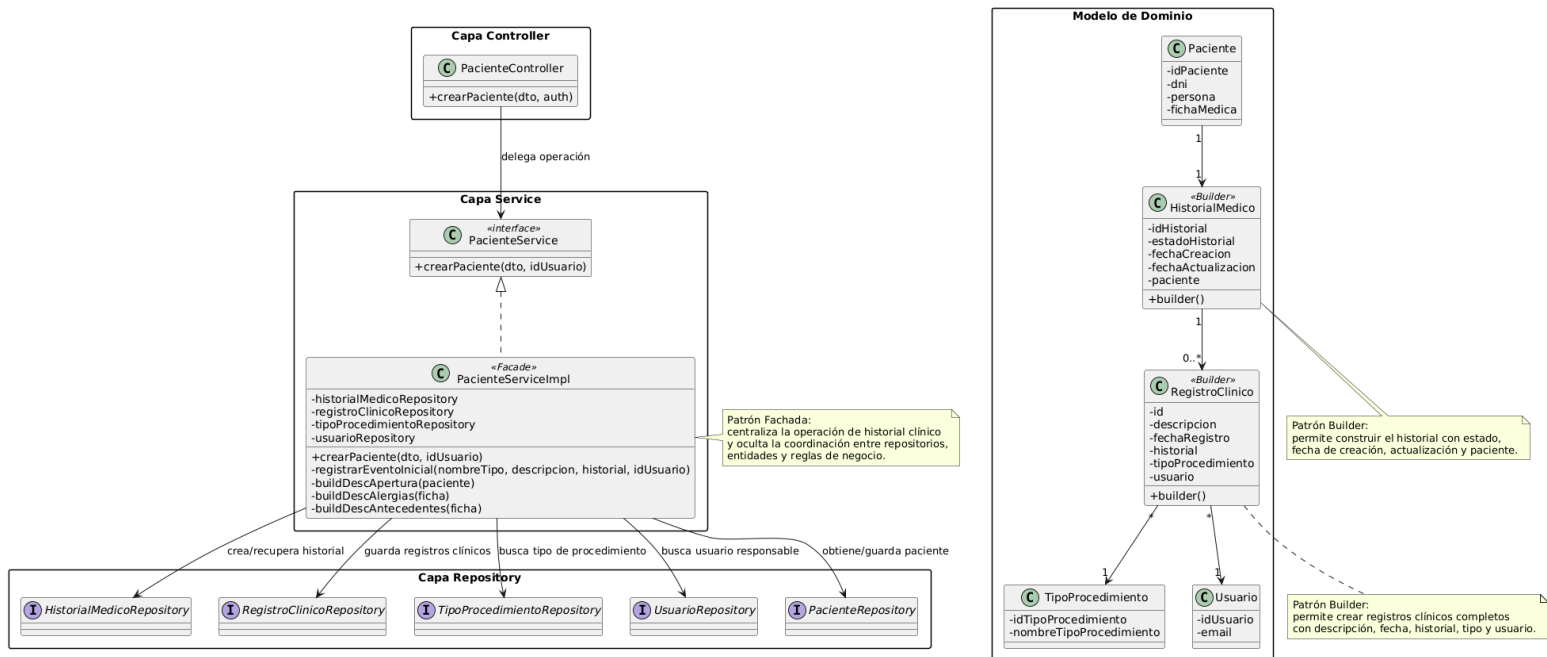


Figura 10: Diagrama representativo de los patrones Fachada y Builder aplicados en Clinicks

El diagrama representa la aplicación de los patrones Fachada y Builder en la funcionalidad de Historial Clínico. La clase **PacienteServiceImpl** cumple el rol de Fachada porque concentra la lógica necesaria para crear o actualizar el historial clínico, registrar eventos iniciales y coordinar el acceso a los repositorios requeridos. De esta manera, el controlador solo invoca una operación del servicio, sin conocer los detalles internos del proceso.

Por otro lado, las entidades **HistorialMedico** y **RegistroClinico** aplican el patrón Builder, ya que se construyen con múltiples atributos y relaciones. Esto permite una inicialización clara, ordenada y controlada de los objetos del dominio, especialmente en operaciones vinculadas con la trazabilidad clínica del paciente.

Capítulo 3. Herramientas y/o lenguajes de programación

3.1. Evaluación de Arquitecturas, Selección y Justificación

Para el desarrollo del sistema Clinicks, se consideró la evaluación de diversas arquitecturas de *software* con el objetivo de seleccionar la que mejor se adapte a los requerimientos de seguridad, escalabilidad y mantenibilidad, aspectos cruciales en un entorno de gestión de datos de salud.

3.1.1 Arquitectura por Capas

Justificación:

Para un sistema donde la protección de datos sensibles es un requisito crítico (Ley 25.326, Ley 26.529), se optó por una arquitectura de capas lógicas (N-Layer) donde las dependencias son estrictamente unidireccionales (cada capa solo conoce a la capa inmediatamente inferior). Esto permite aplicar controles de seguridad en cada frontera, aislar la lógica de negocio de los detalles de persistencia, y garantizar que por ejemplo un cambio en la capa de datos no propague efectos al frontend.

- **Seguridad y Aislamiento de Datos:** La capa de Acceso a Datos (DAL) se aísla de la capa de Presentación, asegurando que la única forma de interactuar con la base de datos sea a través de la Lógica de Negocio. Esto minimiza el riesgo de inyecciones SQL u otros ataques dirigidos a los datos de salud (Historias Clínicas, DNI, diagnósticos, etc.).
- **Mantenibilidad y Escalabilidad:** Permite modificar o actualizar una capa sin afectar significativamente a las demás. Por ejemplo, si se necesita cambiar la base de datos (capa de datos), la Lógica de Negocio permanece intacta, lo que facilita el mantenimiento a largo plazo.
- **Reutilización:** La Lógica de Negocio puede ser reutilizada por diferentes interfaces de usuario (web, móvil) o incluso por otros sistemas en el futuro.

Capas Clave:

1. **Capa de Presentación (UI):** Responsable de la interacción con el usuario (formularios, visualización de historias clínicas).

2. **Capa de Lógica de Negocio (BLL):** Contiene las reglas del negocio (validación de datos de pacientes, lógica de asignación de habitaciones, reglas de registro de eventos clínicos).
3. **Capa de Acceso a Datos (DAL):** Gestiona la conexión y las operaciones CRUD (Crear, Leer, Actualizar, Borrar) con la base de datos.

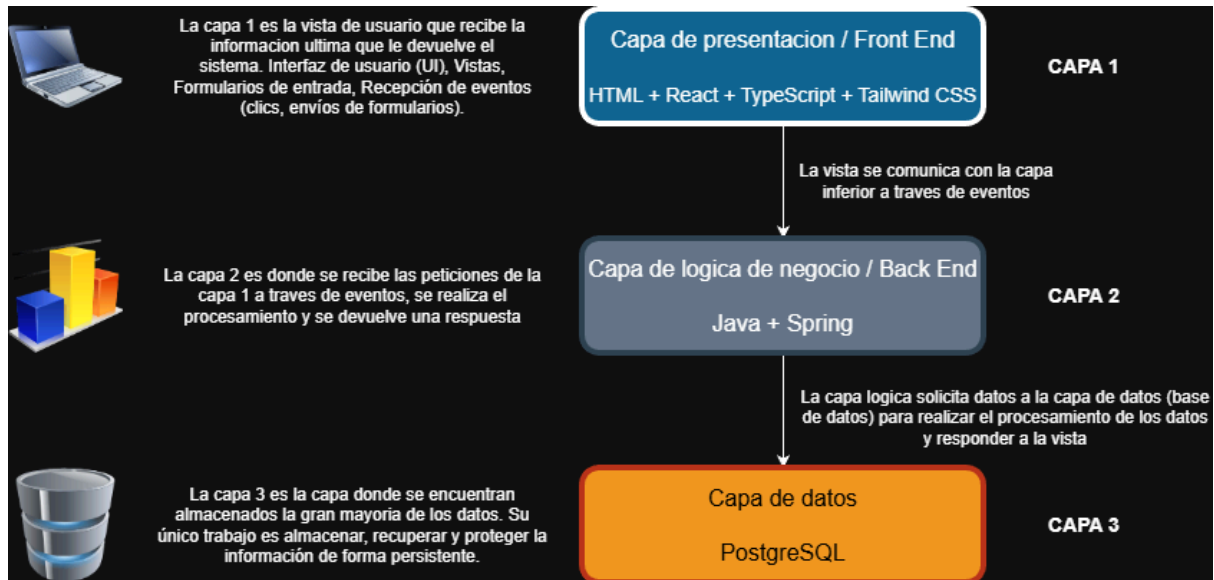


Figura 9: Diagrama de Arquitectura de Capas

3.2. Herramientas utilizadas

Para asegurar la calidad, trazabilidad y escalabilidad del sistema Clinicks, se ha seleccionado un ecosistema de herramientas que abarca desde la gestión del ciclo de vida del *software* hasta el despliegue continuo.

3.2.1. Gestión de Proyecto y Colaboración

Jira (Atlassian): Se utiliza como la herramienta central para la gestión del flujo de trabajo bajo la metodología Scrum. La estructura se organiza mediante la creación de *Epics* (grandes funcionalidades como "Gestión de Pacientes"), *User Stories* (necesidades del usuario) y *Tasks* técnicas. Esto permite realizar una estimación precisa de tiempos y recursos, cumpliendo con la exigencia de la cátedra de documentar el avance de cada integrante.

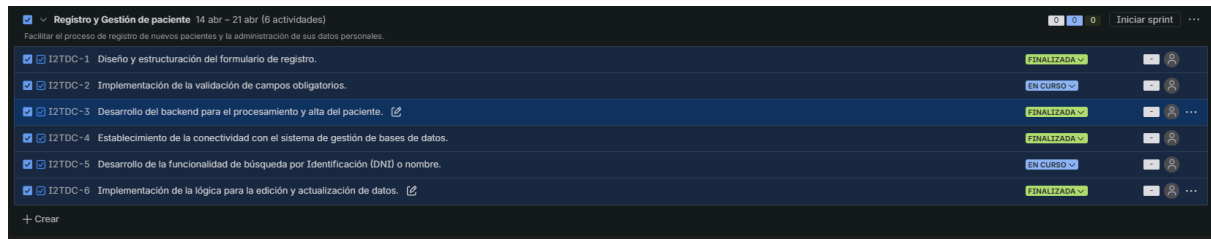


Figura 11: Captura de pantalla de Sprints de Jira

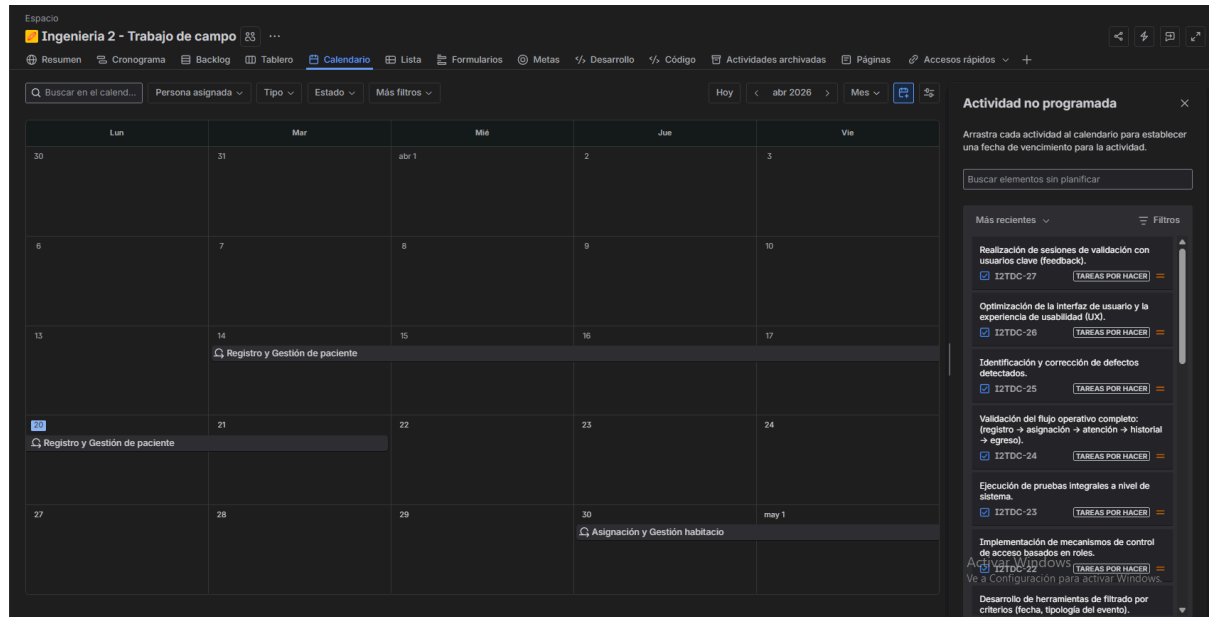


Figura 12: Captura de pantalla de Calendario de Jira

GitHub: Actúa como el repositorio central de código y documentación. La integración con Jira permite vincular cada *commit* con una tarea específica. Se ha definido una estrategia de ramificado (Gitflow) con ramas **main** para producción, **develop** para integración y ramas **feature/** para el desarrollo de funcionalidades individuales, asegurando la integridad del código mediante *Pull Requests* y revisiones de pares.

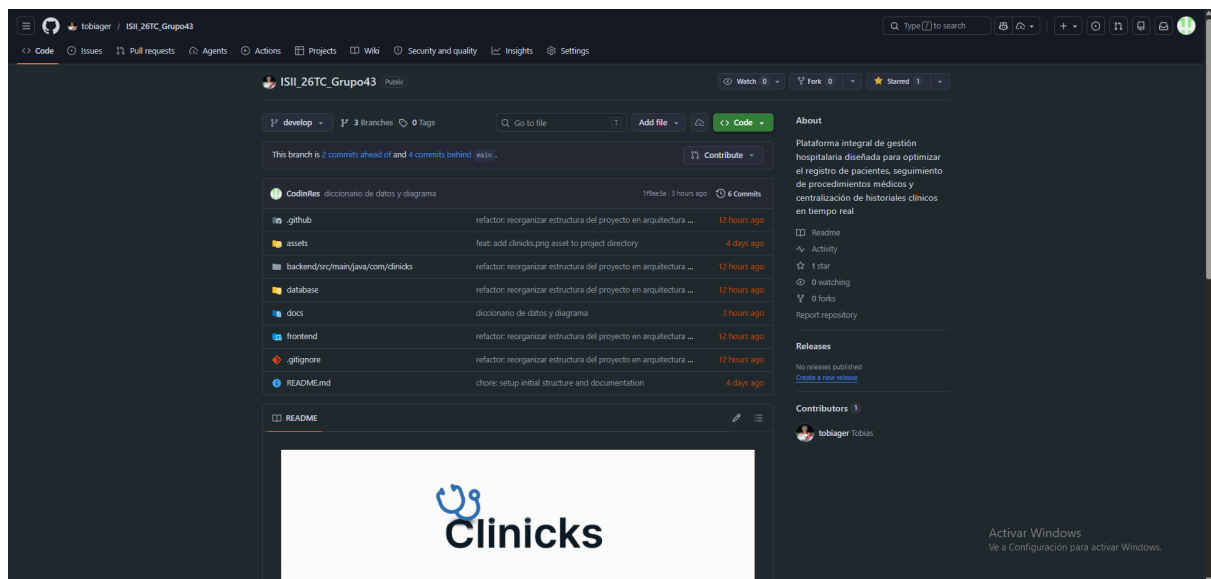


Figura 13: Captura de pantalla del Repositorio de Github

3.2.2. Diseño y Modelado (UML)

Lucidchart: Siguiendo las pautas de la asignatura, se emplean herramientas de diseño formal para la elaboración de los Diagramas de Casos de Uso y Diagramas de Secuencia. Estas representaciones gráficas son esenciales para validar la lógica del negocio antes de la implementación, permitiendo abstraer la complejidad de procesos como la admisión hospitalaria y el manejo de estados en las internaciones.

3.2.3. Stack Tecnológico de Desarrollo

Backend: Spring Boot (Java): Elegido por su robustez en entornos empresariales y su arquitectura basada en capas (*Controller*, *Service*, *Repository*). Esta separación de responsabilidades facilita el mantenimiento y la realización de pruebas unitarias. La persistencia se gestiona mediante *Spring Data JPA*, garantizando que las operaciones de *Soft Delete* en pacientes se realicen de manera consistente a nivel de base de datos.

Se utiliza también Apache Maven que es una herramienta de compilación para proyectos Java. Utilizando un modelo de objetos de proyecto (POM), Maven gestiona la compilación, las pruebas y la documentación de un proyecto.

Frontend: React + TypeScript + Tailwind CSS: La combinación de React y TypeScript proporciona un entorno de desarrollo con tipado fuerte, lo que reduce errores en el tiempo de ejecución. *Tailwind CSS* se utiliza para garantizar una interfaz

de usuario limpia y profesional, optimizando la velocidad de carga mediante la generación de estilos bajo demanda.

Seguridad: Spring Security & JWT: Para proteger los datos sensibles de salud, se implementa autenticación basada en *tokens* JWT (*JSON Web Tokens*) y el *hasheo* de contraseñas mediante BCrypt. Esto permite un control de acceso granular basado en roles (ADMINISTRATIVO, MEDICO, ENFERMERO).

3.2.4. Infraestructura y Persistencia

PostgreSQL (Alojada en Supabase): Como sistema de gestión de bases de datos relacionales, PostgreSQL ofrece la integridad referencial necesaria para un sistema hospitalario. El uso de Supabase permite contar con un entorno en la nube escalable y accesible para todos los integrantes del grupo, facilitando la sincronización de los datos de prueba.

Vercel: Utilizado para el despliegue automático del *frontend*. Cada cambio en la rama principal se refleja inmediatamente en un entorno de producción accesible para los docentes, lo que facilita la verificación de los avances funcionales.

3.2.5. Convenciones y Estándares de Calidad

Para mantener la coherencia técnica, el equipo ha adoptado las siguientes convenciones:

Estandarización Lingüística: El código fuente (clases, métodos, variables) se redacta íntegramente en inglés, siguiendo estándares internacionales, mientras que la documentación y los comentarios se mantienen en español para claridad del equipo y la cátedra.

Trazabilidad y Auditoría: Todas las operaciones críticas en el sistema, especialmente aquellas relacionadas con el historial clínico, generan registros de auditoría que documentan quién realizó la acción y en qué momento, cumpliendo con los principios de integridad y no repudio.

Capítulo 4. Resultados

4.1. Funcionalidad Desarrolladas

Durante el desarrollo del sistema se implementaron las funcionalidades principales previstas para Clinicks, orientadas a la gestión hospitalaria de pacientes, historial clínico, asignación de habitaciones y administración de usuarios. Estas funcionalidades permiten centralizar la información operativa y clínica del paciente, facilitar el trabajo del personal autorizado y mantener la trazabilidad de las acciones realizadas dentro del sistema.

El desarrollo se realizó respetando la arquitectura por capas definida previamente, separando la presentación, la lógica de negocio, la persistencia y la seguridad. De esta manera, las interfaces implementadas se vinculan con servicios backend específicos y entidades persistidas en la base de datos, manteniendo coherencia con el modelo de datos y los casos de uso documentados.

4.1.1. Presentación de la Interfaz: Módulo de Gestión de Pacientes

Se desarrolló la interfaz de usuario (Frontend) para el módulo de Gestión de Pacientes utilizando React y Tailwind CSS, ofreciendo una vista centralizada para las operaciones de alta, consulta y administración de registros.

4.1.1.1 Pantalla Principal - Listado de Pacientes (Dashboard)

Panel Administrativo
Sistema de Gestión Hospitalaria

15 Total Pacientes 0 Internados 7 Camas Disponibles 25 Registros Totales

Pacientes
13 de 13 pacientes, 15 ambulatorios

Ver Desactivados Registrar Paciente

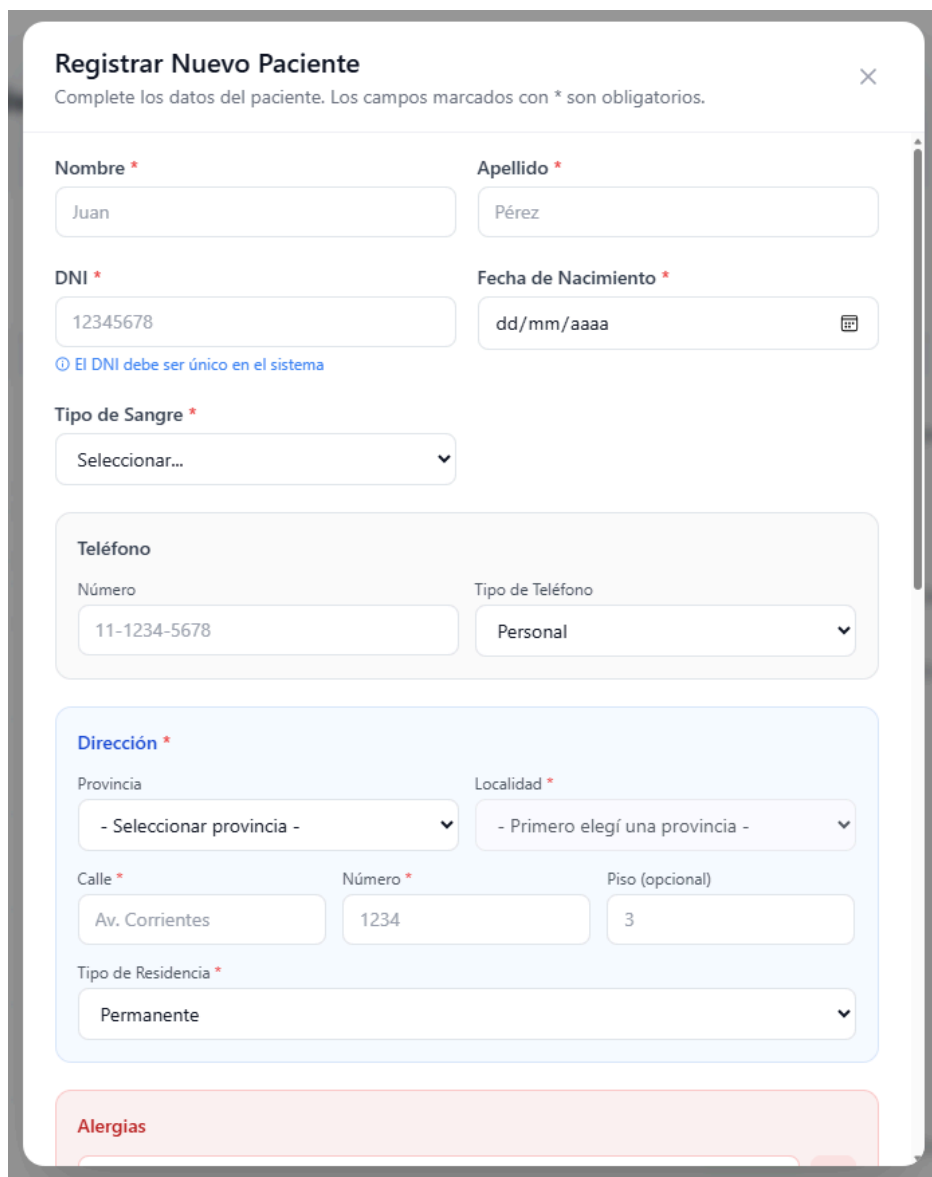
Buscar por nombre o DNI... Todas las Obras Sociales Todos los estados

PACIENTE	DNI	EDAD	TIPO SANGRE	OBRA SOCIAL	ESTADO	ÚLTIMA VISITA	ACCIONES
JC Carlos Juan	11111111	26 años	AB+	PAMI	Ambulatorio	20/04/2026	Ver Editar Eliminar
RD Darin Ricardo Alergias: Látex	30000003	69 años	B-	PAMI	Ambulatorio	—	Ver Editar Eliminar
LE Espósito Lali Alergias: Aspirina	30000008	34 años	B+	PAMI	Ambulatorio	—	Ver Editar Eliminar
MG García Maria Alergias: Polen	30000002	33 años	O+	Swiss Medical	Ambulatorio	—	Ver Editar Eliminar
BG Gonzalo Bizarrap	30000007	27 años	O+	Swiss Medical	Ambulatorio	—	Ver Editar Eliminar
FL Lopez Fermin	34232345	26 años	O-	—	Ambulatorio	20/04/2026	Ver Editar Eliminar

Figura 14: Captura de pantalla del Dashboard con lista de pacientes, filtros y opción 'Ver Desactivados'.

Descripción: La interfaz muestra una tabla con los pacientes activos, incluyendo filtros por DNI y nombre, y un botón de acción para registrar un nuevo paciente.

4.1.1.2. Formulario de Alta/Modificación de Paciente



Registrar Nuevo Paciente

Complete los datos del paciente. Los campos marcados con * son obligatorios.

Nombre * Apellido *

Juan Pérez

DNI * Fecha de Nacimiento *

12345678 dd/mm/aaaa

El DNI debe ser único en el sistema

Tipo de Sangre *

Seleccionar...

Teléfono

Número Tipo de Teléfono

11-1234-5678 Personal

Dirección *

Provincia Localidad *

- Seleccionar provincia - - Primero elegí una provincia -

Calle * Número * Piso (opcional)

Av. Corrientes 1234 3

Tipo de Residencia *

Permanente

Alergias

Figura 15: Captura de pantalla del formulario de registro de paciente, mostrando DNI, Nombre, Apellido, Fecha de Nacimiento.

Registrar Nuevo Paciente

Complete los datos del paciente. Los campos marcados con * son obligatorios.

Sin alergias registradas

Enfermedades Crónicas

Ej: Diabetes tipo 2

+

Sin enfermedades crónicas registradas

Antecedentes Familiares

Ej: Hipertensión

+

Sin antecedentes familiares registrados

Observaciones — Antecedentes Familiares

Descripción libre de antecedentes familiares relevantes...

Contactos de Emergencia

+ Agregar

Sin contactos de emergencia registrados

Obra Social

Seleccionar

Nro. Afiliado

Sin obra social

▼

OSDE-001234

[El Nro. de Afiliado debe ser único por Obra Social](#)

Figura 16: Captura de pantalla del formulario de registro de paciente, mostrando Alergias, Enfermedades Crónicas, Antecedentes Familiares, Observaciones, Contacto de Emergencias y Obra Social

Descripción: El formulario se estructura para capturar todos los datos definidos en el Diccionario de Datos (Sección 2.6.2), incluyendo información personal (persona, dni), de contacto, domicilio (residencia, domicilio) y datos de cobertura (**afiliacion_obra_social**). La validación de campos sigue el **schema** de validación documentado en la conversación (Sección 2.5.1).

4.1.1.3 Acción de Baja Lógica (Soft Delete)

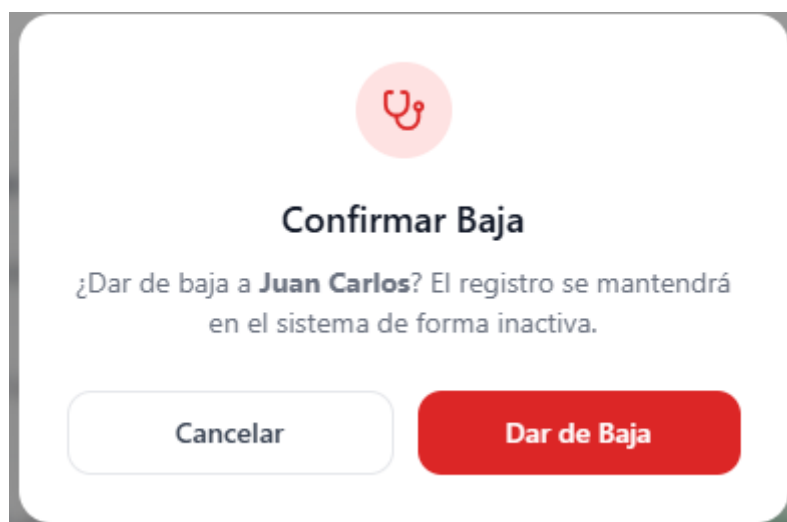


Figura 17: Captura de pantalla de la alerta de confirmación previa a la baja lógica

Descripción: Al intentar "dar de baja" un paciente, se invoca un modal de confirmación para prevenir eliminaciones accidentales. Una vez confirmada, el sistema actualiza el campo **deleted_at** sin borrar el registro físico, cumpliendo con el criterio de aceptación GP-05 del Product Backlog.

4.1.1.4. Listado de Pacientes Desactivados

o

x

Pacientes Desactivados

Registros dados de baja del sistema

PACIENTE	DNI	EDAD	OBRA SOCIAL	ACCIONES
WA Wanchope Abila	12345678	87 años	IOMA	Restaurar

1 paciente desactivado

Figura 18: Captura de pantalla Lista de Pacientes Desactivados

Descripción: Esta vista muestra únicamente los registros que tienen completado el campo **deleted_at**. Los usuarios autorizados pueden consultar esta lista para verificar el historial de bajas y, en caso necesario, reincorporar al paciente al sistema activo.

4.1.2. Presentación de la Interfaz: Módulo de Historial Clínico

El módulo de Historial Clínico permite consultar y registrar la evolución clínica de un paciente a partir de eventos ordenados cronológicamente. Esta funcionalidad responde al alcance definido para el sistema, donde se establece que cada paciente debe contar con un expediente único, registros clínicos asociados, trazabilidad del profesional responsable y visualización en formato de línea de tiempo.

4.1.2.1 Pantalla Principal - Listado de Pacientes (Dashboard)

Historial Médico

55 pacientes

FILTROS

Q

Nombre, apellido o DNI...

Estado: Todos

Obra social: Todas

☐

Con alergias

PACIENTE	DNI	EDAD	ESTADO	OBRA SOCIAL	ÚLTIMA VISITA	
BA Acosta, Brenda Alergias: Ácaros	46789123	23 años	Ambulatorio	—	05/05/2005	Ver historial
AA Aguirre, Ana Alergias: Sulfas	35456789	34 años	Ambulatorio	Medifé	24/01/2024	Ver historial
JA Alderete, Jorge Alergias: Penicilina	18888124	62 años	Ambulatorio	PAMI	20/09/2014	Ver historial
SA Arias, Santiago	47777123	21 años	Ambulatorio	—	17/04/2023	Ver historial
EB Barrios, Enrique Alergias: Sulfas	21111124	58 años	Ambulatorio	PAMI	24/02/2010	Ver historial
HB Benítez, Héctor Alergias: Látex	23888999	52 años	Ambulatorio	IOMA	11/11/2025	Ver historial
EC Cabrera, Emanuel	33333123	37 años	Ambulatorio	Jerárquicos Salud	06/02/2026	Ver historial
JC Cáceres, Julieta Alergias: Ibuprofeno	40123456	29 años	Ambulatorio	Jerárquicos Salud	21/10/2009	Ver historial
RC Cano, Raúl Alergias: Penicilina	23333123	54 años	Ambulatorio	IOMA	11/12/2021	Ver historial

Figura 19: Captura de pantalla del listado de pacientes con historial médico.

Descripción: La pantalla muestra el módulo de **Historial Médico**, donde se presenta un listado de pacientes registrados en el sistema. La interfaz permite buscar pacientes por nombre, apellido o DNI, filtrar por estado, obra social y presencia de alergias. Para cada paciente se visualizan datos relevantes como DNI, edad, estado, obra social y fecha de última visita. Además, cada registro cuenta con la acción **“Ver historial”**, que permite acceder al historial clínico individual del paciente seleccionado.

4.1.2.2 Pantalla de informacion del Historial del Paciente

← Volver al historial

ALERGIAS: PENICILINA

JA

Alderete, Jorge Ambulatorio

62 años · DNI 18888124 · A+ · PAMI · Afiliado AF-0042-18888124
Tierra del Fuego 930, Posadas, Misiones · Residencia permanente

Datos médicos complementarios 2 enf. crónicas 2 antec. familiares

Historial Clínico

16 registros en total

+ Registrar Evento

FILTROS

Buscar en registros...

Seleccionar fechas

Tipo: Todos

Origen: Todos

Profesional: Todos

INTERNACIONES (1)

Internación #1 Con alta

Hab. 304 — Piso 3 · 19 nov 2011 · 00:00 — 24 nov 2011 · 00:00 · 7 eventos

ATENCIONES AMBULATORIAS (9)

GUARDIA

Consulta por guardia. Se indican pautas de alarma y control.

21 sep 2014 · 05:00 · Laura Enfermera ENFERMERO

DERIVACIÓN

Derivación a especialidad para evaluación complementaria

Figura 19: Captura de pantalla del módulo de Historial Clínico del paciente.

Descripción: La pantalla muestra la información principal del paciente seleccionado y su historial clínico organizado en una línea de tiempo. Cada registro permite identificar la fecha, el tipo de procedimiento, la descripción clínica y el profesional responsable de la carga.

4.1.2.3 Formulario para registrar nuevo evento clínico

The screenshot shows a web interface for a medical history system. At the top, there's a header with a calendar icon, the text 'Historial Clínico', and '16 registros en total'. A blue button labeled '+ Registrar Evento' is in the top right. Below this is a light blue box titled 'Nuevo evento clínico'. Inside, there's a label 'Tipo de procedimiento' above a dropdown menu with the placeholder 'Seleccionar tipo...'. To the right, there's a label 'Descripción' above a large text input area with the placeholder 'Descripción del evento clínico...'. At the bottom left of the box are two buttons: 'Cancelar' and 'Guardar evento'.

Figura 20: Captura de pantalla del formulario para registrar un nuevo evento clínico.

Descripción: Desde el historial del paciente, los usuarios autorizados pueden registrar un nuevo evento clínico indicando el tipo de procedimiento y una descripción. Al confirmar la operación, el sistema asocia automáticamente el registro al historial médico del paciente y al usuario autenticado, garantizando la trazabilidad de la intervención. Esta lógica se corresponde con el caso de uso “Registrar evento clínico”, donde el sistema valida los datos, identifica al profesional responsable y actualiza la línea de tiempo luego de guardar el registro .

4.1.2.4 Línea de tiempo de Historial Clínico

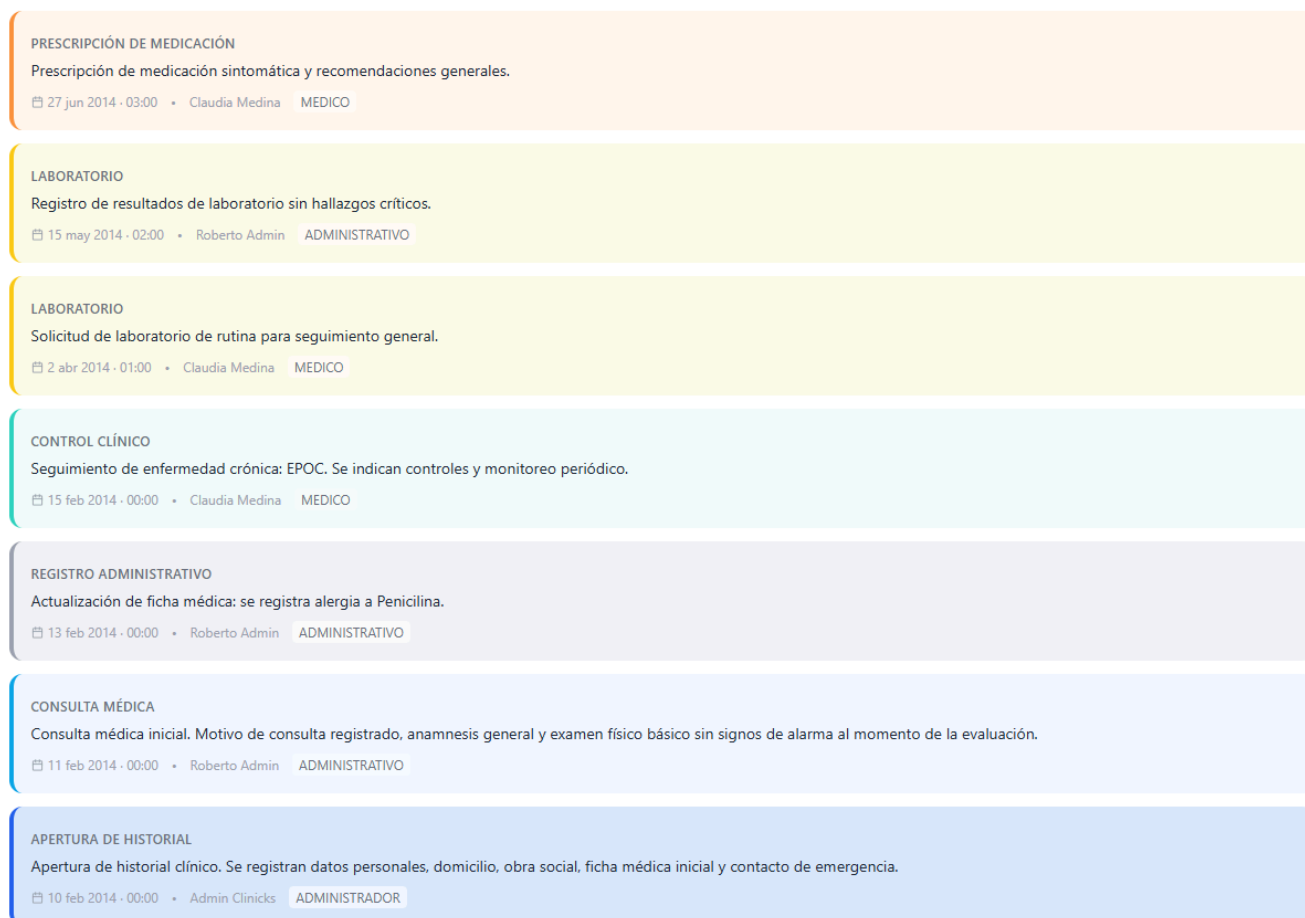


Figura 21: Captura de pantalla de la línea de tiempo del Historial Clínico.

Descripción: La línea de tiempo permite reconstruir la evolución del paciente a partir de sus registros clínicos. Los eventos se presentan ordenados por fecha, tipo de procedimiento, descripción y profesional interviniente, coincidiendo con el caso de uso “Visualizar línea de tiempo de historial clínico” documentado previamente .

4.1.3. Presentación de la Interfaz: Asignación y Gestión de Habitaciones

Se implementó el módulo de gestión de habitaciones con el objetivo de administrar la disponibilidad de camas y habitaciones del hospital. Esta funcionalidad permite visualizar habitaciones disponibles, ocupadas o en mantenimiento, seleccionar un paciente e iniciar una internación cuando corresponda.

4.1.3.1 Pantalla de Habitaciones

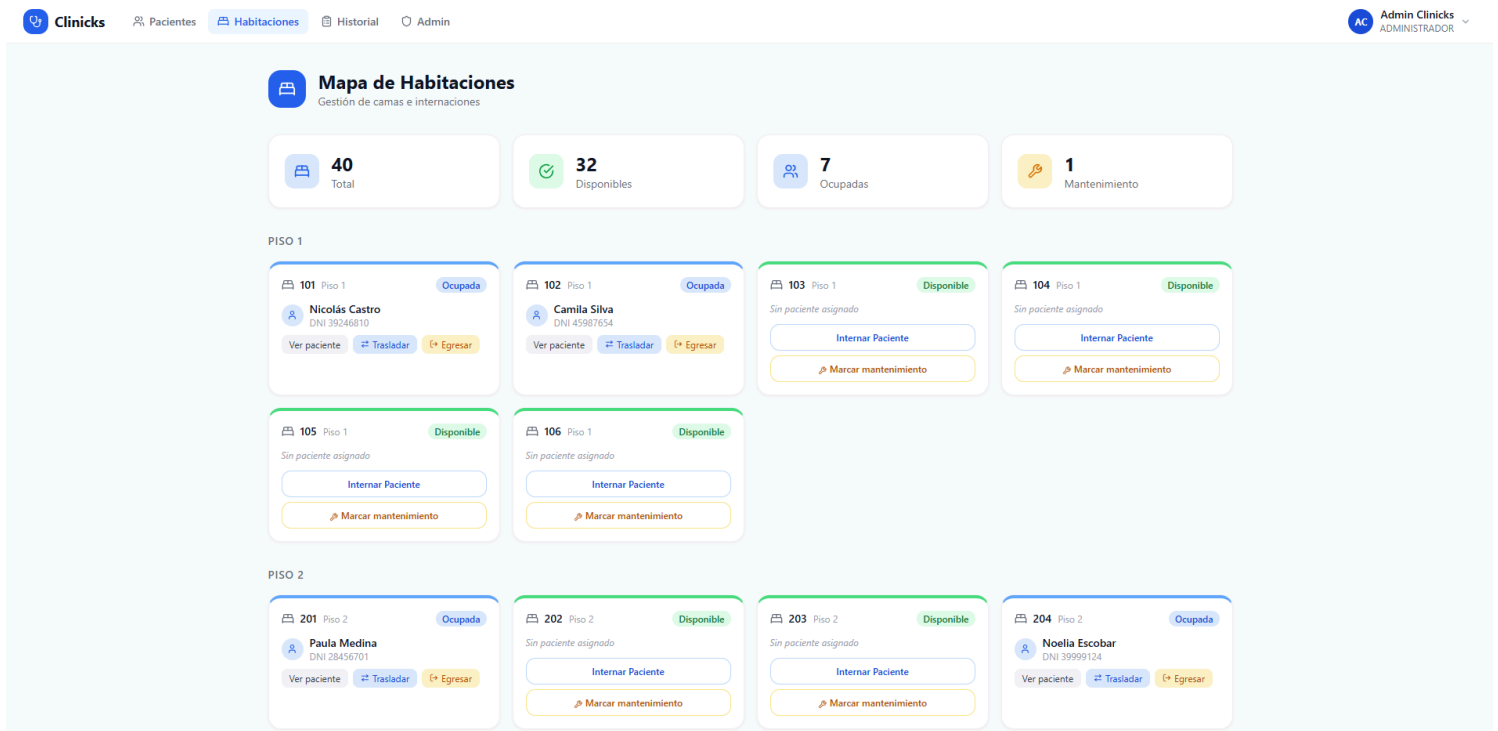
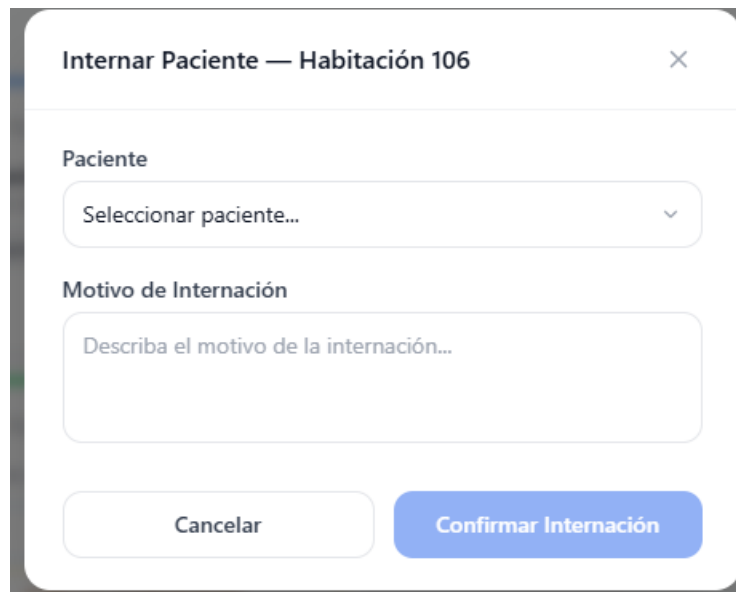


Figura 22: Captura de pantalla del listado de habitaciones por piso y estado.

Descripción: La interfaz presenta las habitaciones organizadas visualmente, permitiendo identificar rápidamente su disponibilidad. Las habitaciones ocupadas no permiten iniciar una nueva internación, evitando duplicidades y errores operativos.

4.1.3.2 Formulario para asignar Habitación a un Paciente



The screenshot shows a mobile application interface for assigning a patient to a room. The title bar at the top reads 'Internar Paciente — Habitación 106' with a close button (X) on the right. Below the title bar, there are two main sections. The first section is labeled 'Paciente' and contains a dropdown menu with the text 'Seleccionar paciente...' and a downward arrow. The second section is labeled 'Motivo de Internación' and contains a text input field with the placeholder text 'Describe el motivo de la internación...'. At the bottom of the form, there are two buttons: a white button with a gray border labeled 'Cancelar' and a blue button labeled 'Confirmar Internación'.

Figura 23: Captura de pantalla de la asignación de habitación a un paciente.

Descripción: Al seleccionar una habitación disponible, el sistema permite elegir un paciente existente y completar los datos requeridos para la internación, como motivo y observaciones. Una vez confirmada la operación, se registra la internación, se actualiza el estado de la habitación y se genera el evento clínico correspondiente en el historial del paciente.

Esta funcionalidad se encuentra alineada con el alcance inicial del sistema, donde se definió la necesidad de registrar ingresos y egresos, asignar habitaciones y visualizar la disponibilidad de habitaciones dentro del hospital . Además, en los casos de prueba ya se valida que una habitación disponible pueda asignarse correctamente, que no se pueda internar en habitaciones ocupadas y que no se permita duplicar una internación activa para el mismo paciente .

4.1.4. Presentación de la Interfaz: Panel de Administración de Usuarios

Se incorporó un panel de administración destinado al usuario con rol administrador. Este módulo permite gestionar los usuarios del sistema y controlar el acceso según los roles definidos para el personal hospitalario. La funcionalidad resulta necesaria para mantener la seguridad del sistema, ya que Clinicks contempla acceso restringido mediante autenticación y roles para médicos, enfermeros y administrativos .

4.1.4.1 Pantalla Principal - Listado de Usuarios (Dashboard)

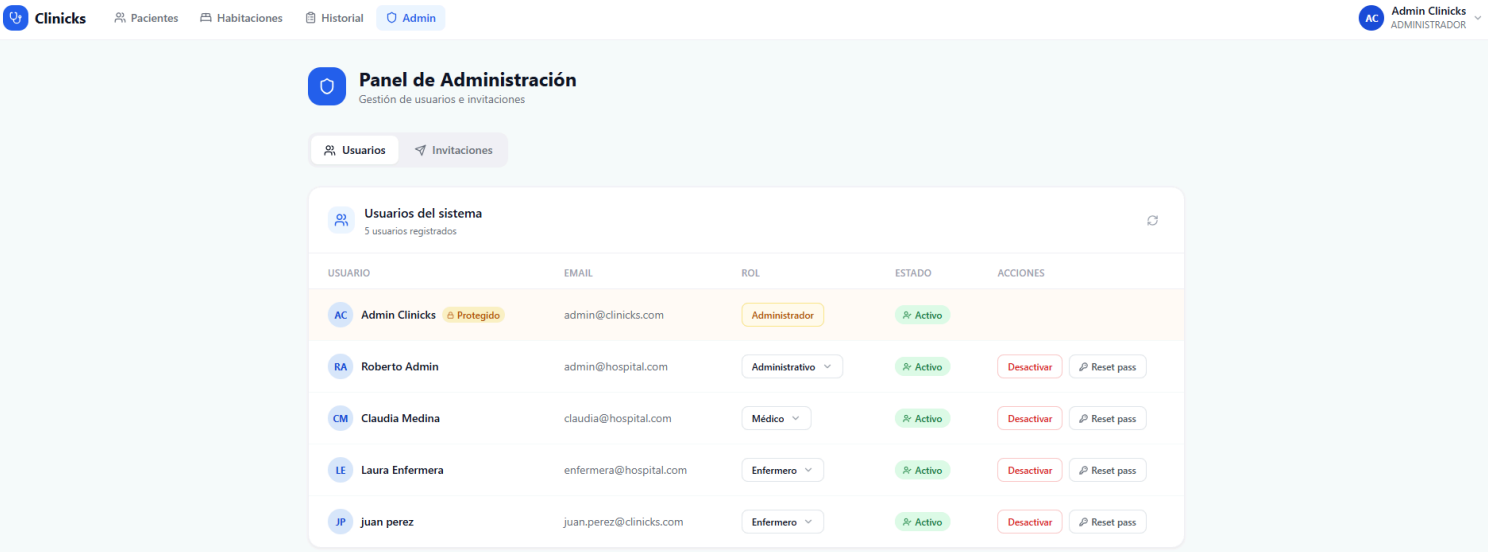


Figura 24: Captura de pantalla del Panel de Administración de Usuarios.

Descripción: La pantalla permite visualizar los usuarios registrados, consultar sus datos principales, identificar su rol dentro del sistema y administrar su estado. Esta vista centraliza las operaciones de control de usuarios, facilitando la gestión interna del personal autorizado.

4.1.4.2 Pantalla para generar invitacion de registro a un nuevo usuario

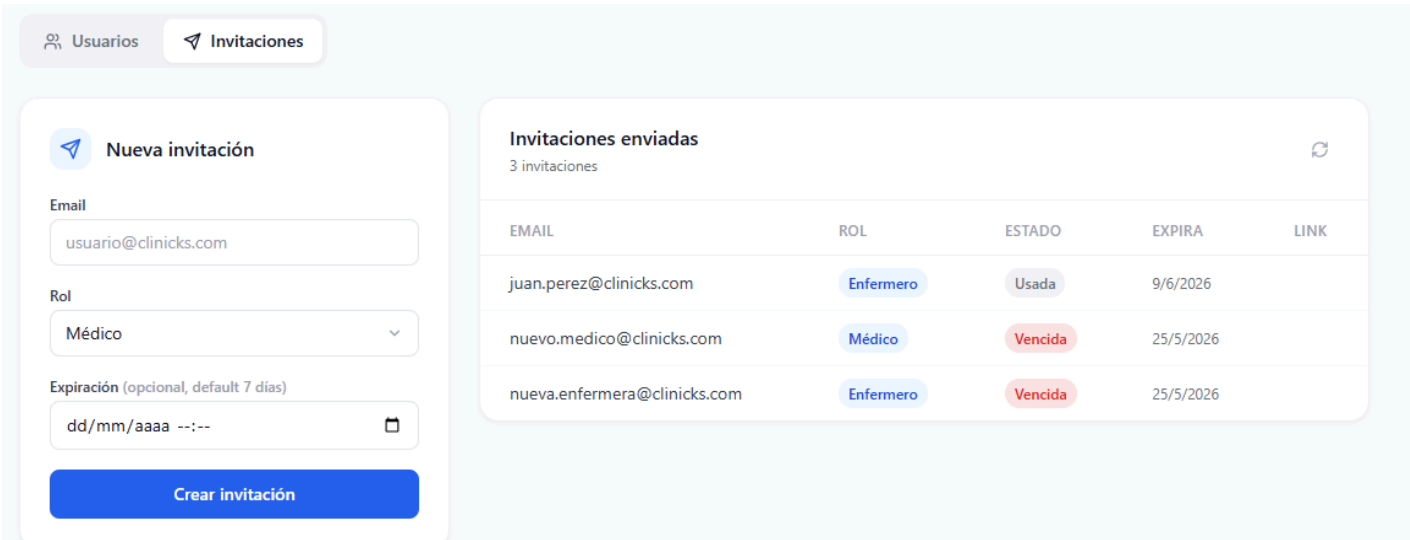


Figura 25: Captura de pantalla de la creación de invitación para registro de usuario.

Descripción: El administrador puede generar una invitación para que un nuevo usuario se registre en el sistema. Esta invitación permite controlar el alta de usuarios,

evitando registros abiertos o no autorizados. Al crear la invitación, el sistema define los datos necesarios para que el nuevo usuario complete su registro y quede vinculado al rol correspondiente.

Esta funcionalidad mejora el control de acceso y permite mantener una administración segura de los perfiles del sistema. Además, deja trazabilidad sobre el proceso de incorporación de nuevos usuarios, lo cual resulta coherente con las necesidades de seguridad, roles y auditoría del sistema.

4.2. Aspectos Técnicos del Backend y Persistencia

En esta sección se detalla la implementación lógica que soporta la interfaz de usuario del módulo de pacientes, asegurando la integridad de los datos y el cumplimiento de la arquitectura por capas definida en la sección 3.1.1.

4.2.1. Lógica de Negocio y Desacoplamiento (DTOs)

Para la comunicación entre el Frontend (React) y el Backend (Spring Boot), se implementó el patrón **DTO (Data Transfer Object)**.

- **Propósito:** Evitar la exposición directa de las entidades de la base de datos a la capa de presentación. Esto permite que, por ejemplo, al registrar un paciente, el sistema reciba un objeto **PacienteRequestDTO** que solo contiene los campos necesarios para la creación, protegiendo atributos sensibles o internos del servidor.
- **Validaciones:** Se utiliza la especificación Jakarta Bean Validation (anotaciones como `@NotNull`, `@Size`, `@Email`) en los DTOs. Esto garantiza que ningún dato inconsistente llegue a la base de datos, rechazando peticiones mal formadas directamente en el controlador.

4.2.2. Estrategia de Persistencia y Integridad

La persistencia se gestiona mediante **Spring Data JPA** sobre una base de datos **PostgreSQL**. Se destaca el cumplimiento de los siguientes puntos técnicos:

- **Manejo de Relaciones:** Al persistir un nuevo paciente, el sistema gestiona de forma transaccional la inserción en las tablas relacionadas (domicilio, telefono y ficha_medica), asegurando que no existan registros huérfanos y manteniendo la normalización definida en el DER (Sección 2.6.1).

- **Mapeo de Tipos de Datos:** Se respetaron estrictamente los tipos de datos definidos en el Diccionario de Datos (Sección 2.6.2), utilizando tipos ENUM para campos con opciones cerradas (como tipos de sangre o sexo) y TIMESTAMP para registros de auditoría.

4.2.3. Implementación de Baja Lógica (Soft Delete)

Siguiendo las buenas prácticas de sistemas de salud, no se realizan eliminaciones físicas (HARD DELETE) en la tabla de pacientes.

- **Mecánica:** Se añadió un atributo `deleted_at` de tipo TIMESTAMP en la entidad paciente.
- **Lógica:** Cuando el usuario selecciona "Eliminar", el servicio de backend marca este campo con la fecha actual en lugar de borrar la fila. Las consultas de "Pacientes Activos" filtran automáticamente aquellos registros donde `deleted_at` sea nulo.
- **Ventaja:** Esto permite mantener la trazabilidad de los datos para futuras auditorías médicas y facilita la recuperación de registros desactivados por error sin pérdida de integridad referencial.

4.2.4. Implementación del Historial Clínico

La implementación del historial clínico se apoya en las entidades `HistorialMedico`, `RegistroClinico` y `TipoProcedimiento`. Cada paciente posee un historial asociado, dentro del cual se registran los eventos clínicos generados durante su atención. Los registros clínicos se almacenan con fecha, descripción, tipo de procedimiento y usuario responsable.

Para garantizar la integridad de la información, los registros clínicos no se eliminan físicamente ni se modifican libremente, ya que representan evidencia de la atención realizada. Esta decisión permite conservar la trazabilidad médica y mantener un registro cronológico de la evolución del paciente.

4.2.5. Implementación de Internaciones y Habitaciones

La asignación de habitaciones se implementó mediante la relación entre pacientes, habitaciones e internaciones. Al registrar una internación, el sistema verifica que la

habitación exista, se encuentre disponible y que el paciente no posea una internación activa. Luego, se actualiza el estado de la habitación y se registra la internación correspondiente.

Esta validación evita inconsistencias como asignar dos pacientes a la misma habitación o internar dos veces al mismo paciente en simultáneo. Además, la operación queda vinculada al usuario que la ejecuta, fortaleciendo la trazabilidad administrativa.

4.2.6. Implementación de Seguridad, Roles e Invitaciones

El sistema utiliza autenticación y control de acceso basado en roles. El panel de administración permite gestionar usuarios y generar invitaciones de registro para nuevos integrantes del sistema. Este mecanismo evita que cualquier persona pueda registrarse libremente y permite que el administrador controle previamente qué usuarios podrán acceder.

Las invitaciones de registro se almacenan en la base de datos y se vinculan al proceso de alta de usuarios. De esta manera, el sistema mantiene un flujo controlado para la incorporación de médicos, enfermeros, administrativos y administradores.

4.3. Casos de prueba

4.3.1 Casos de prueba de funcionalidad “Login”

Versión del caso de prueba: 1

Autor del caso de prueba: Samuel Zini

Fecha de creación: 17/5

Caso de uso: Login

Nombre del probador: Samuel Zini

Fecha de ejecución: 18/5

CP	Objetivo	Datos de entrada	Resultado esperado	Resultado obtenido
1	Ingresar al sistema con credenciales válidas	Email: lunaperla@hotmail.com · Contraseña: Clave123!	El sistema autentica correctamente, genera token y devuelve el usuario autenticado.	

2	Ingresar al sistema con email inexistente	Email: inexistente@clinicks.com · Contraseña: Clave123!	El sistema rechaza el acceso con credenciales inválidas.	
3	Ingresar al sistema con email válido y contraseña incorrecta	Email: juan.perez@clinicks.com · Contraseña: ClaveIncorrecta	El sistema rechaza el acceso con credenciales inválidas.	
4	Ingresar al sistema con email y contraseña inválidos	Email: otro@clinicks.com · Contraseña: 123456	El sistema rechaza el acceso con credenciales inválidas.	
5	Ingresar al sistema con email y contraseña vacíos	Email: vacío · Contraseña: vacío	El sistema no permite continuar y devuelve errores de validación.	
6	Ingresar al sistema con email vacío y contraseña válida	Email: vacío · Contraseña: Clave123!	El sistema no permite continuar y devuelve error de validación en el email.	
7	Ingresar al sistema con email válido y contraseña vacía	Email: juan.perez@clinicks.com · Contraseña: vacío	El sistema no permite continuar y devuelve error de validación en la contraseña.	
8	Ingresar al sistema con email y contraseña solo con espacios	Email: " " · Contraseña: " "	El sistema no permite continuar y devuelve errores de validación.	
9	Ingresar al sistema con email con espacios laterales	Email: " juan.perez@clinicks.com " · Contraseña: Clave123!	El sistema rechaza el email por formato inválido.	
10	Ingresar al sistema con credenciales largas	Email largo y contraseña larga	El sistema valida el formato y rechaza	
11	Ingresar al sistema con contraseña con caracteres especiales	Email: juan.perez@clinicks.com · Contraseña: ' OR '1'='1	El sistema no autentica si la contraseña no coincide.	

4.3.2 Casos de prueba de funcionalidad "Registrar Paciente"

Versión del caso de prueba: 2

Autor del caso de prueba: Samuel Zini

Fecha de creación: 17/5

Caso de uso: Registrar Paciente

Nombre del probador: Samuel Zini

Fecha de ejecución: 18/5

CP	Objetivo	Datos de entrada	Resultado esperado	Resultado obtenido
1	Registrar un paciente con datos válidos	Nombre: Sole · Apellido: Stella · DNI: 45678901 · Fecha de nacimiento: 1990-05-15 · Tipo de sangre: A+ · Teléfono: 1123456789 · Tipo de teléfono: personal · Dirección: Corrientes · Número: 1234 · Piso: 3 · Localidad: 1 · Tipo de residencia: permanente	El sistema registra correctamente al paciente y devuelve respuesta exitosa.	Pasó. Se creó el registro
2	Registrar un paciente sin campos obligatorios	Nombre: vacío · Apellido: vacío · DNI: vacío · Fecha de nacimiento: vacío · Tipo de sangre: vacío	El sistema rechaza el envío y muestra errores de validación.	

3	Registrar un paciente con fecha de nacimiento futura	Fecha de nacimiento: mañana	El sistema rechaza la fecha por no ser válida.	
4	Registrar un paciente con tipo de sangre inválido	Tipo de sangre: X+	El sistema rechaza el valor por no pertenecer a las opciones permitidas.	
5	Registrar un paciente con tipo de teléfono inválido	Tipo de teléfono: hogar	El sistema rechaza el valor por no pertenecer a las opciones permitidas.	
6	Registrar un paciente con tipo de residencia inválido	Tipo de residencia: fijo	El sistema rechaza el valor por no pertenecer a las opciones permitidas.	
7	Registrar un paciente con DNI duplicado	DNI: 12345678	El sistema no permite duplicar el DNI.	
8	Registrar un paciente con teléfono ya existente	Teléfono: igual a otro ya existente	El sistema no permite repetir el teléfono en el sistema.	
9	Registrar un paciente con teléfono repetido en contacto de emergencia	Contacto de emergencia con teléfono igual al principal o repetido	El sistema no permite repetir teléfonos dentro de la misma solicitud.	
10	Registrar un paciente con número de afiliado duplicado	Obra social y número de afiliado ya existentes	El sistema no permite duplicar el número de afiliados para esa obra social.	Fallo, las verificaciones y la introducción de los números no está normalizado en la base de datos así que se pudo crear dos con igual número de afiliación pero diferente formato

11	Registrar un paciente normalizando nombre y apellido	Nombre: JUAN carlos · Apellido: del VALLE	El sistema capitaliza correctamente nombre y apellido antes de guardar.	
----	--	--	---	--

Casos de prueba de funcionalidad “Registrar Paciente”

4.3.3 Casos de prueba de funcionalidad “Asignar Habitaciones”

Versión del caso de prueba: 1
 Autor del caso de prueba: Samuel Zini
 Fecha de creación: 17/5
 Caso de uso: Asignar Habitaciones
 Nombre del probador: Samuel Zini
 Fecha de ejecución: 18/5

CP	Objetivo	Datos de entrada	Resultado esperado	Resultado obtenido
1	Asignar una habitación disponible a un paciente existente	idHabitacion: 1 · Paciente: Nicolas Castro · motivo: Dolor abdominal	El sistema registra la internación, marca la habitación como ocupada y genera el evento clínico.	
2	Asignar una habitación disponible aunque motivo y observaciones estén vacíos	idHabitacion: 4 · Paciente: Del Valle Juan Carlos · motivo: vacío o espacios · observaciones: vacío	El sistema no permite aceptar el alta porque el motivo es campo obligatorio.	
3	Intentar asignar una habitación que ya está ocupada	idHabitacion: 1 · Paciente: Cáceres Julieta	El sistema no debería dar la opción a internar en una habitación ocupada	
4	Intentar asignar una habitación a un paciente inexistente	idHabitacion: 5 · Paciente: 999	El sistema solo puede asignar habitación a pacientes existentes	

5	Intentar asignar una habitación a un paciente que ya tiene internación activa	idHabitacion: 1 · Paciente: Nicolas Castro con internación activa	El sistema no permite duplicar la internación del paciente.	
6	Intentar asignar una habitación inexistente	idHabitacion: 999 · Paciente: Nicolas Castro	El sistema rechaza la operación porque la habitación no existe.	

Casos de prueba de funcionalidad “Asignar Habitaciones”

4.4. Pruebas Unitarias

4.4.1 Prueba unitaria: Método: crearPaciente(PacienteRequestDTO dto, Integer idUsuario)

Nota

Este método registra un nuevo paciente en el sistema. Recibe un DTO con los datos personales, médicos, de contacto y residencia del paciente, junto con el ID del usuario que lo registra.

Variables de Entrada							
nombre	apellido	dni	fechaNacimiento	tipoSangre	telefono	Descripción	Resultado esperado
"juan carlos"	"del valle"	12345678	1990-05-15	"A+"	"1123456789"	Registro exitoso con datos mínimos válidos	Se crea el paciente y se devuelve un PacienteResponseDTO con DNI 12345678
"JUAN carlos"	"del VALLE"	12345678	1990-05-15	"A+"	"1123456789"	Registro con nombre en mayúsculas/mi núsculas mixtas; se verifica la normalización	Se crea el paciente con nombre "Juan Carlos" y apellido "Del Valle"

"juan carlos"	"del valle"	12345678 (ya existente)	1990-05-15	"A+"	"1123456789"	Intento de registro con DNI ya existente en el sistema	Se lanza DniDuplicadoException con mensaje que contiene "12345678"
"juan carlos"	"del valle"	12345678	1990-05-15	"A+"	"1123456789" (ya existente en tabla teléfono)	Intento de registro con teléfono ya registrado como teléfono personal de otro paciente	Se lanza TelefonoDuplicadoException con mensaje que contiene "1123456789"
"juan carlos"	"del valle"	12345678	1990-05-15	"A+"	"1123456789" (ya existente en contacto emergencia)	Intento de registro con teléfono ya registrado como contacto de emergencia de otro paciente	Se lanza TelefonoDuplicadoException
"juan carlos"	"del valle"	12345678	1990-05-15	"A+"	"1123456789" + contactos con tel repetido "1187654321" x2	Intento de registro con dos contactos de emergencia que tienen el mismo número de teléfono	Se lanza TelefonoDuplicadoException con mensaje que contiene "1187654321"
"juan carlos"	"del valle"	12345678	1990-05-15	"A+"	"1123456789" + obraSocial con nroAfiliado "ABC123" (duplicado)	Intento de registro con número de afiliado ya existente para la misma obra social	Se lanza AfiliadoDuplicadoException con mensaje que contiene "ABC123"
"juan carlos"	"del valle"	12345678	1990-05-15	"A+"	"1123456789" + dirección null	Registro sin dirección, se verifica que use valor por defecto	Se crea el paciente con dirección "Sin dirección"

4.4.2 Prueba unitaria: Método: `internarPaciente(Integer idHabitacion, InternacionRequestDTO dto, Integer idUsuario)`

Nota

Este método registra la internación de un paciente en una habitación. Recibe el ID de la habitación, un DTO con el ID del paciente, motivo y observaciones, y el ID del usuario que ejecuta la acción.

Variables de Entrada							
idHabitacion	idPaciente	Motivo	Observaciones	Estado habitación	Estado paciente	Descripción	Resultado esperado
1	10	"Dolor abdominal"	"Se asigna por observación"	Disponible	No internado	Internación exitosa con todos los datos válidos	La habitación pasa a estado "ocupada", se crea registro de internación y evento en historial
2	10	"Dolor abdominal"	"Se asigna por observación"	Ocupada	No internado	Intento de internación en habitación ocupada	Se lanza <code>HabitacionNoDisponibleException</code> : "La habitación 102 no está disponible."
3	10	"Dolor abdominal"	"Se asigna por observación"	Mantenimiento	No internado	Intento de internación en habitación en mantenimiento	Se lanza <code>HabitacionNoDisponibleException</code> : "La habitación 103 no está disponible."
1	10	"Dolor abdominal"	null	Disponible	No internado	Internación exitosa sin observaciones (campo opcional)	Se asigna la habitación correctamente. La habitación pasa a "ocupada", se crea registro de internación y evento en historial sin observaciones

1	10 (inexistente)	"Dolor abdominal"	"Observación"	Disponible	—	Intento de internación del paciente que no existe en el sistema	Se lanza <code>PacienteNoEncontradoException</code> con mensaje que contiene "10"
1	10	"Dolor abdominal"	"Observación"	Disponible	Ya internado	Intento de internación de paciente que ya está internado en otra habitación	Se lanza <code>OperacionNoPermitidaException</code> : "El paciente ya se encuentra internado."
1 (inexistente)	10	"Dolor abdominal"	"Observación"	—	No internado	Intento de internación en habitación que no existe en el sistema	Se lanza <code>RuntimeException</code> : "Habitación no encontrada"
1	10	"Control post-op"	"Paciente estable"	Disponible	No internado, sin historial previo	Internación de paciente que no tiene historial médico previo	Se crea un nuevo historial médico automáticamente, se asigna la habitación y se registra el evento

Capítulo 5. Conclusiones y futuros trabajos

5.1 Conclusiones

A partir del desarrollo del sistema Clinicks, se logró construir una solución web orientada a la gestión hospitalaria, enfocada principalmente en la administración de pacientes, historiales clínicos, internaciones, habitaciones y usuarios del sistema. El trabajo permitió aplicar de manera integrada los contenidos abordados durante la asignatura Ingeniería del Software II, incluyendo planificación, análisis de requisitos, modelado, diseño de arquitectura, implementación, pruebas y documentación.

El sistema desarrollado permite centralizar información relevante de los pacientes, registrar datos personales, antecedentes médicos, obra social, contactos de emergencia y estado administrativo. Además, incorpora funcionalidades vinculadas con la asignación de habitaciones, el control de disponibilidad y el registro de internaciones, evitando inconsistencias como asignar una misma habitación a más de un paciente o internar dos veces al mismo paciente en simultáneo. Estas validaciones fueron contempladas tanto en la lógica de negocio como en los casos de prueba documentados.

Uno de los principales resultados alcanzados fue la implementación del historial clínico digital, el cual permite registrar eventos clínicos asociados a cada paciente, organizados cronológicamente en una línea de tiempo. Cada registro queda vinculado al usuario responsable, al tipo de procedimiento y a la fecha correspondiente, fortaleciendo la trazabilidad de la información médica. Esta decisión resulta especialmente importante en un sistema de salud, donde la integridad y conservación de los datos son aspectos críticos.

También se implementó un esquema de seguridad basado en autenticación y roles, permitiendo diferenciar los permisos de administradores, médicos, enfermeros y administrativos. El panel de administración de usuarios y el sistema de invitaciones permiten controlar el acceso al sistema, evitando registros no autorizados y manteniendo una administración más segura de los perfiles internos.

Desde el punto de vista técnico, el proyecto fue desarrollado con una arquitectura por capas, utilizando React con TypeScript para el frontend, Spring Boot para el backend y PostgreSQL como base de datos. El repositorio del proyecto confirma esta composición tecnológica, con predominio de TypeScript, Java y PLpgSQL, además de una estructura dividida en frontend, backend, database y documentación. Esta organización favoreció la separación de responsabilidades, la mantenibilidad del código y la posibilidad de extender el sistema en futuras versiones.

En cuanto a la gestión del trabajo, el uso de GitHub, Jira y una metodología ágil basada en Scrum permitió organizar las tareas por sprints, distribuir responsabilidades y mantener trazabilidad entre la planificación, la documentación y la implementación. El repositorio cuenta con carpetas específicas para frontend, backend, base de datos y

documentación, además de un historial de commits que evidencia la evolución del desarrollo .

En conclusión, Clinicks cumple con los objetivos principales planteados para el trabajo de campo, ya que ofrece una base funcional para la gestión hospitalaria y permite demostrar la aplicación práctica de conceptos de Ingeniería del Software. Si bien el sistema todavía puede ampliarse, la versión desarrollada constituye una solución coherente, funcional y alineada con las necesidades definidas para el proyecto.

5.2 Aprendizajes obtenidos

Durante el desarrollo del proyecto se reforzó la importancia de realizar una correcta planificación antes de implementar funcionalidades. La definición de casos de uso, diagramas de secuencia, contratos de operaciones, modelo de datos y casos de prueba permitió reducir ambigüedades y mantener una relación más clara entre los requisitos y el código desarrollado.

También se evidenció la relevancia de la trazabilidad en sistemas que manejan información sensible. En Clinicks, esta trazabilidad se aplicó mediante registros clínicos vinculados a usuarios, fechas y tipos de procedimientos, así como mediante baja lógica para preservar información histórica de pacientes. Estas decisiones permitieron mejorar la confiabilidad del sistema y evitar pérdidas de información.

Otro aprendizaje importante fue la necesidad de mantener una arquitectura ordenada. La separación entre controladores, servicios, repositorios, DTOs y entidades facilitó el mantenimiento del backend y permitió implementar validaciones de negocio de forma más clara. Del mismo modo, el uso de componentes en el frontend ayudó a construir una interfaz más organizada y reutilizable.

5.3 Dificultades encontradas

Durante el proyecto surgieron dificultades relacionadas con la integración entre frontend, backend y base de datos, especialmente al trabajar con entidades relacionadas como paciente, ficha médica, domicilio, obra social, historial clínico e internación. Estas relaciones exigieron validar correctamente los datos y cuidar la persistencia transaccional para evitar registros incompletos o inconsistentes.

Otra dificultad fue definir correctamente los permisos de cada rol, ya que el sistema debía contemplar diferentes tipos de usuarios con responsabilidades distintas. Para resolverlo, se implementó una lógica de autenticación y autorización que permite restringir funcionalidades según el perfil del usuario.

Además, la implementación del historial clínico requirió tomar decisiones sobre la conservación de los registros. Se optó por no permitir la eliminación física de eventos clínicos históricos, priorizando la trazabilidad y la integridad de la información.

5.4 Futuros trabajos

Como futuras líneas de desarrollo, se propone ampliar Clinicks incorporando funcionalidades que complementen la gestión hospitalaria actual. Entre las mejoras posibles se destacan:

Implementar un módulo de turnos médicos, que permita programar consultas, asignar profesionales y registrar asistencia de pacientes.

Agregar un módulo de prescripción médica, donde los profesionales puedan indicar medicamentos, dosis, frecuencia y duración del tratamiento.

Incorporar gestión de stock de medicamentos e insumos hospitalarios, vinculando el uso de recursos con procedimientos o internaciones.

Desarrollar un sistema de notificaciones internas, para alertar al personal sobre nuevas internaciones, altas, cambios de habitación o eventos clínicos relevantes.

Implementar reportes estadísticos sobre ocupación de habitaciones, cantidad de pacientes internados, frecuencia de procedimientos y evolución de altas.

Mejorar la interoperabilidad del sistema mediante la adopción de estándares de intercambio de información sanitaria, como HL7 FHIR, permitiendo una futura integración con laboratorios, centros de diagnóstico u otros sistemas hospitalarios.

Fortalecer la seguridad incorporando auditorías más detalladas, registro de accesos, historial de cambios y políticas avanzadas de permisos.

Ampliar las pruebas automatizadas, incluyendo pruebas de integración y pruebas end-to-end para validar el comportamiento completo del sistema desde la interfaz hasta la base de datos.

Finalmente, se propone mejorar la experiencia de usuario mediante pruebas con usuarios reales o simulados del ámbito hospitalario, a fin de ajustar formularios, flujos de navegación y criterios de usabilidad según las necesidades concretas del personal médico, administrativo y de enfermería.

Bibliografía consultada

- [1] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7ma ed. México: McGraw-Hill, 2010.
- [2] K. Schwaber y J. Sutherland, "La Guía de Scrum: Las Reglas del Juego," Scrum.org, nov. 2020. [En línea]. Disponible en: <https://scrumguides.org/>
- [3] Meta Platforms Inc., "React Documentation," 2024. [En línea]. Disponible en: <https://react.dev/>
- [4] VMware Tanzu, "Spring Boot Reference Documentation," 2024. [En línea]. Disponible en: <https://docs.spring.io/spring-boot/documentation.html>
- [5] PostgreSQL Global Development Group, "PostgreSQL 16.0 Documentation," 2023. [En línea]. Disponible en: <https://www.postgresql.org/docs/16/index.html>
- [6] Supabase Inc., "Supabase Documentation," 2024. [En línea]. Disponible en: <https://supabase.com/docs>
- [7] Atlassian, "Documentación de Jira Software," 2024. [En línea]. Disponible en: <https://www.atlassian.com/es/software/jira/guides>